Spec and test gaps: hmac

Repository `targets/opentitan` at commit `b84154c71810`. Report generated 2026-11-10 21:06 UTC by chipbag.

Each finding below gives the gap, the evidence for it as exact quotes at their file and line, and how it was verified: an agent searched the repository to refute the finding, and the finding was kept only if that failed. Every quote was checked against the file at the cited lines. The report does not recommend a fix.

Summary

#	Category	Verified	Withdrawn	Inconclusive
1	Spec too ambiguous to test	11	8	0
2	Spec contradicts itself	1	1	0
3	Test contradicts its own description	2	5	0
4	Spec and test disagree	0	0	0
5	Spec not tested	32 (94 items)	17	3

2 verified findings with the same subject as another are shown together with it and counted once.

Scope

Spec sources (5): `hw/ip/hmac/README.md`, `hw/ip/hmac/data/hmac.hjson`, `hw/ip/hmac/doc/interfaces.md`, `hw/ip/hmac/doc/programmers_guide.md`, `hw/ip/hmac/doc/theory_of_operation.md`

Verification sources (54 files): `hw/ip/hmac/dv/env/seq_lib/` (14), `sw/device/tests/` (8), `hw/dv/tools/dvsim/testplans/` (5), `hw/ip/hmac/dv/env/` (5), `hw/dv/sv/test_vectors/vectors/hmac/` (4), `hw/ip/hmac/data/` (2), `hw/ip/hmac/dv/` (2), `hw/ip/hmac/dv/cov/` (2), `hw/top_earlgrey/dv/` (2), `sw/device/tests/crypto/` (2), `hw/ip/hmac/doc/` (1), `hw/ip/hmac/dv/cryptoc_dpi/` (1), `hw/ip/hmac/dv/sva/` (1), `hw/ip/hmac/dv/tests/` (1), `hw/ip/hmac/model/` (1), `hw/top_earlgrey/data/ip/` (1), `hw/top_earlgrey/dv/env/seq_lib/` (1), `sw/device/lib/dif/` (1). The full list is in the manifest.

411 requirements after merging duplicates; 338 need a test (the rest hold by construction). 88 tests (68 with source), 65 test plan items. 210 requirements are matched to a test.

Files the verifier read that are not in the manifest: `hw/dv/sv/csr_utils/csr_hw_reset_seq.sv`, `hw/dv/tools/dvsim/testplans/csr_testplan.hjson`, `hw/dv/tools/dvsim/tests/csr_tests.hjson`, `hw/dv/tools/dvsim/tests/tl_access_tests.hjson`, `hw/ip/hmac/README.md`, `hw/ip/hmac/data/hmac.hjson`, `hw/ip/hmac/doc/programmers_guide.md`, `hw/ip/hmac/doc/registers.md`, `hw/ip/hmac/dv/env/hmac_scoreboard.sv`, `hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv`, `hw/ip/hmac/dv/env/seq_lib/hmac_smoke_vseq.sv`, `hw/ip/prim/doc/prim_packer.md`, `sw/device/tests/hmac_enc_test.c`, `sw/device/tests/hmac_error_conditions_test.c`, `sw/device/tests/keymgr_dpe_sideload_hmac_test.c`.

1. Spec too ambiguous to test

1.1 The module can wipe internal variables such as the secret key, intermediate hash results, digest and message scheduling array.

`f_08abef58a364` · vague · severity medium · section: hw/ip/hmac/README.md > Overview > Description

Gap: The 'such as' list does not state exactly which internal variables are wiped.

Spec says:

- `hw/ip/hmac/README.md:59`: `is able to wipe internal variables such as the secret key, intermediate hash results, digest and the internal message scheduling array.`

Not answered by the spec: Exactly which state does a WIPE_SECRET write overwrite with the written value? Please confirm each of: key registers, intermediate hash state, all DIGEST_0..15 registers regardless of digest_size, message schedule array, message length counter (MSG_LENGTH), and any other internal state. Which software-visible registers must then read back the wipe value, replicated per 64-bit word for SHA-2 384/512? And what is the effect on a hash that is in progress?

Verified: No other spec text enumerates exactly which state WIPE_SECRET overwrites. The register description and README repeat the same non-exhaustive "such as" list. They add only that the 32-bit value is replicated for 64-bit SHA-2 384/512 words, and that the message FIFO is not wiped. They say nothing about MSG_LENGTH, the saved-context registers, or the effect on a hash in progress.

Further evidence found by the verifier:

- `hw/ip/hmac/data/hmac.hjson:455-457`: `The clear secret operation overwrites the internal variables with the provided 32-bit value.` — The register description uses the same "such as" list and does not name MSG_LENGTH, DIGEST_0..15 for every digest size, or in-flight behaviour.

Searched:

- wipe / WIPE_SECRET in hmac md and hjson in `hw/ip/hmac`: Only README.md:59-63, the hmac.hjson WIPE_SECRET desc and the generated registers.md; all give the same non-exhaustive list.

Looked at and ruled out:

- `hw/ip/hmac/README.md:62-63`: It states the replication for 64-bit words, which partly answers the question, but it does not give the scope of the wipe or the in-flight behaviour.

1.2 The position of these zeros depends on the endianness, thus on the programmed CFG.key_swap.

`f_2ab2710b6d0c` · missing detail · severity medium · section: hw/ip/hmac/data/hmac.hjson > hmac > CFG

Gap: The position of the padding zeros for each key_swap value is not stated.

Spec says:

- `hw/ip/hmac/data/hmac.hjson:311`: `The position of these zeros depends on the endianness, thus on the programmed CFG.key_swap.`

Not answered by the spec: For a key shorter than the programmed key_length, at which byte positions of the KEY_* registers (and of the last partial word) must the zero padding sit when key_swap=0 and when key_swap=1?

Verified: Other HMAC docs only repeat the same sentence. None says where the padding zeros sit in the KEY_* registers or the last partial word for key_swap=0 versus 1. The finding stands.

Further evidence found by the verifier:

- `hw/ip/hmac/doc/theory_of_operation.md:125`: `The position of these zeros depends on the endianness, thus on the programmed [`CFG.key_swap`](registers.md#cfg--key_swap).` — Repeats the same statement with no byte positions.
- `hw/ip/hmac/doc/registers.md:234`: `If 1 the endianness of each KEY_* register is swapped. Default value (value 0) is big endian representation of the KEY_* CSRs.` — Describes the swap only; gives no padding position.

Searched:

- key_swap, padding and zero-pad in the HMAC docs in `hw/ip/hmac (*.md)`: Only the same sentence repeated in registers.md and theory_of_operation.md. Other padding mentions concern SHA-2 message padding and ipad/opad.

Looked at and ruled out:

- `hw/ip/hmac/doc/theory_of_operation.md:123`: Says the key registers are big-endian by default and can be swapped, but does not say where the zero padding sits.

Same subject, also found as spec too ambiguous to test (`f_7cebd576a1f3`, missing detail): The zero position for each CFG.key_swap value is not specified.

Verified: The same sentence appears in registers.md (CFG.key_length) and in hmac.hjson. No document gives the KEY_n word or byte positions of the zero padding for either key_swap value. The only other text says key_swap swaps the endianness of each KEY_* register and that the default is big endian. That does not say which KEY words hold the key bits and which hold zeros.

Further evidence found by the verifier:

- `hw/ip/hmac/doc/registers.md:197-198`: `The position of these zeros depends on the endianness, thus on the programmed [`CFG.key_swap`](registers.md#cfg--key_swap).` — Repeats the same vague sentence in the register description. The 80-bit example gives no register layout.

1.3 they are irrelevant and should not be read out.

`f_1f0bad59c809` · weak modal · severity medium · section: hw/ip/hmac/data/hmac.hjson > hmac > DIGEST

Gap: 'should not be read out' does not say what a read returns or whether it is prohibited.

Spec says:

- `hw/ip/hmac/data/hmac.hjson:504`: `they are irrelevant and should not be read out.`

Not answered by the spec: In SHA-2 384 mode, what value does a read of DIGEST12-DIGEST15 return (zero, the truncated internal state words, or undefined), and does the read complete without a bus error?

Verified: No spec text defines what a read of DIGEST12-15 returns in SHA-384 mode. It also doesn't say whether the read causes a bus error. The register docs repeat the same wording. The programmer's guide only says the registers are not to be read and hold irrelevant values.

Further evidence found by the verifier:

- `hw/ip/hmac/doc/programmers_guide.md:44`: `For SHA-2 384, only \`DIGEST_0\`..\`11\` should be read out, the rest should be truncated out by not being read by the SW.` — Only says software should not read them. It defines no value and no error.

Searched:

- irrelevant, truncat, DIGEST12-15 wording in `hw/ip/hmac docs and hjson`: Only the same wording in registers.md, programmers_guide.md, theory_of_operation.md and hmac.hjson. None defines a read value or whether the read returns an error.

Looked at and ruled out:

- `hw/ip/hmac/doc/programmers_guide.md:43`: Says the unused registers 'would hold irrelevant values'. That is still undefined, so it does not say what a read returns.

1.4 and {DIGEST8 - DIGEST15} are irrelevant and should not be read out.

`f_c9d86b452dad` · weak modal · severity medium · section: hw/ip/hmac/data/hmac.hjson > hmac > DIGEST

Gap: 'should not be read out' does not say what a read returns or whether it is prohibited.

Spec says:

- `hw/ip/hmac/data/hmac.hjson:503`: `and {DIGEST8 - DIGEST15} are irrelevant and should not be read out.`

Not answered by the spec: In SHA-2 256 mode, what value does a read of DIGEST8-DIGEST15 return (zero, stale value, or undefined), and does the read complete without a bus error?

Verified: No other spec text says what a read of DIGEST8-15 returns in SHA-256 mode or whether it errors. The programmer's guide only says the registers "would hold irrelevant values" and should not be read. Zero is specified only while an operation is in progress, which is a different case.

Further evidence found by the verifier:

- `hw/ip/hmac/doc/programmers_guide.md:43`: `For SHA-2 256, only \`DIGEST_0\`..\`7\` should be read out, the remaining digest registers are redundant and would hold irrelevant values.` — Still says 'irrelevant values' and gives no read value or error behaviour.

Searched:

- DIGEST and irrelevant across hmac md/hjson docs in `hw/ip/hmac`: Only the 'irrelevant / should not be read out' wording plus the in-progress-returns-zero statements; nothing on stale or zero value for DIGEST8-15 after SHA-256 completion, and nothing on bus errors.

Looked at and ruled out:

- `hw/ip/hmac/doc/theory_of_operation.md:161`: Covers the zero-read only during a sideloaded-key operation, not the idle SHA-256 case.

1.5 The lower 3 bits [2:0] of the message length are ignored.

`f_dfa447ca7429` · unclear referent · severity medium · section: hw/ip/hmac/data/hmac.hjson > hmac > MSG_LENGTH_LOWER

Gap: It is unclear whether the ignored bits are ignored on software writes, in the hardware length computation, or in the value read back.

Spec says:

- `hw/ip/hmac/data/hmac.hjson:529`: `Lower 3 bits [2:0] are ignored.`

Not answered by the spec: When software writes MSG_LENGTH_LOWER with bits [2:0] nonzero, does readback return those bits or zero? Are bits [2:0] always zero in the hardware-updated value, and are they excluded when the length is used in the padding computation?

Verified: No other spec text says whether bits [2:0] of MSG_LENGTH_LOWER read back as written or as zero. Nothing says whether hardware-updated values always have them clear, or whether the padding computation excludes them. The only related text is "Message is byte granularity", which implies the bits carry no information but doesn't settle the readback behavior.

Searched:

- message length, msg_length, MSG_LENGTH in the HMAC markdown docs in `hw/ip/hmac/**/*.md`: registers.md only repeats the hjson text. theory_of_operation.md covers padding and the length in general terms but says nothing about bits [2:0]. programmers_guide.md covers context save and restore without mentioning the ignored bits.

Looked at and ruled out:

- `hw/ip/hmac/doc/theory_of_operation.md:59-77`: Describes padding and length computation generally. It doesn't say what happens to bits [2:0] on write or readback.

1.6 hmac_idle indicates HMAC idle status.

`f_2f0738731f9b` · missing detail · severity medium · section: hw/ip/hmac/data/hmac.hjson > hmac > STATUS

Gap: The conditions that define the HMAC as idle are not stated.

Spec says:

- `hw/ip/hmac/data/hmac.hjson:417`: `HMAC idle status.`

Not answered by the spec: What is the exact set of conditions under which STATUS.hmac_idle reads 1? Specifically: is it 0 while the message FIFO holds data before hash_start/hash_process? Is it 0 from hash_stop until the stop completes? Which events cause the 0-to-1 transition, and at what latency relative to the hmac_done interrupt?

Verified: No spec text defines the exact conditions under which hmac_idle reads 1. The docs only say writes to DIGEST and MSG_LENGTH are allowed when idle, and that polling hmac_idle can replace waiting for hmac_done after hash_process. Neither answers the FIFO, hash_stop or latency questions.

Further evidence found by the verifier:

- `hw/ip/hmac/data/hmac.hjson:418`: `When IDLE, the \`DIGEST\` and the \`MSG_LENGTH_LOWER\`/\`MSG_LENGTH_UPPER\` can be written to from SW` — Only states the consequence of idle, not its definition.

Searched:

- idle mentions in hmac docs and hjson in `hw/ip/hmac`: Only the register description, the programmer's guide polling note, and a sideload-abort note that says the core returns to idle. No definition of idle conditions.

Looked at and ruled out:

- `hw/ip/hmac/doc/theory_of_operation.md:156`: Says the core returns to idle after a sideload-key abort. It doesn't define hmac_idle for the FIFO, hash_stop or latency cases.

1.7 A software/CPU write of a 32-bit value to WIPE_SECRET wipes the internal variables, including the secret key, intermediate hash results, digest and message scheduling array.

`f_0ee4675cde56` · missing detail · severity medium · section: hw/ip/hmac/data/hmac.hjson > hmac > WIPE_SECRET

Gap: "Some internal variables such as" does not enumerate exactly which variables are cleared.

Spec says:

- `hw/ip/hmac/README.md:61`: `The software can wipe the internal variables and secret key by writing a 32-bit random value into WIPE_SECRET register.`
- `hw/ip/hmac/data/hmac.hjson:455`: `If the CPU writes a value into the register, the value is used to clear some internal variables such as the secret key, intermediate hash results, digest and internal message scheduling array.`

Not answered by the spec: What is the exhaustive list of state a WIPE_SECRET write overwrites: key, digest, hash intermediate state, message schedule, message FIFO, message length, partial block? For each item, is it set to the written value (or its duplicate for 64-bit modes), and what is the defined effect on an in-progress hash and on later DIGEST readback and KEY usage?

Verified: No spec text enumerates the wiped state or its resulting values. The README only adds that the message FIFO is not wiped. Nothing says what digest, key or in-progress hash look like after the wipe. The testplan only says the scoreboard checks that digest data are corrupted.

Searched:

- wipe / WIPE_SECRET in hmac md and hjson in `hw/ip/hmac`: README, hjson, registers.md, testplan. None defines the exact wiped set, the resulting values, or the effect on an in-progress hash.

Looked at and ruled out:

- `hw/ip/hmac/README.md:60`: It answers only the message FIFO part of the question.

1.8 For SHA-2 256, only DIGEST_0..7 should be read out, the remaining digest registers are redundant and would hold irrelevant values.

`f_8fed7e244a4e` · weak modal · severity medium · section: hw/ip/hmac/doc/programmers_guide.md > Programmer's Guide > Triggering HMAC/SHA-2 engine

Gap: 'should be read out' and 'irrelevant values' do not state a testable requirement on the hardware.

Spec says:

- `hw/ip/hmac/doc/programmers_guide.md:43`: `For SHA-2 256, only DIGEST_0..7 should be read out, the remaining digest registers are redundant and would hold irrelevant values.`

Not answered by the spec: In SHA-2 256 mode, what value must DIGEST_8..15 read back after a hash completes (zero, unchanged/stale, or undefined and excluded from checking)? Does this also hold after switching from a SHA-2 384/512 operation?

Verified: No other spec text defines the value of DIGEST_8..15 in SHA-256 mode. The register description repeats that they are "irrelevant and should not be read out", which leaves the value undefined. It doesn't say what they hold after a prior SHA-512 operation.

Further evidence found by the verifier:

- `hw/ip/hmac/data/hmac.hjson:503`: `{DIGEST8 - DIGEST15} are irrelevant and should not be read out.` — Register description: says the registers are irrelevant but gives no value.

Searched:

- DIGEST_8, irrelevant, redundant in hmac md and hjson files in `hw/ip/hmac`: Only the same 'irrelevant / should not be read out' wording. No value is specified.

Looked at and ruled out:

- `hw/ip/hmac/doc/registers.md:422-423`: This is generated from the hjson and repeats the 'irrelevant' wording. It doesn't define a value.

1.9 Starting an operation while the sideloaded key is invalid is blocked.

`f_fc00646d2273` · missing detail · severity medium · section: hw/ip/hmac/doc/programmers_guide.md > Programmer's Guide > Using a sideloaded key

Gap: 'Blocked' does not say how the start command is handled.

Spec says:

- `hw/ip/hmac/doc/programmers_guide.md:110`: `Starting while the key is invalid is blocked`

Not answered by the spec: When CMD.hash_start is written while the sideloaded key is invalid, is the command dropped with the engine left fully idle? Specifically: - Are subsequent MSG_FIFO writes discarded or accepted, and is MSG_LENGTH updated? - Is a following CMD.hash_process ignored, or does it also

report an error? - Are DIGEST_0-15 left unchanged or cleared? - Is the only side effect ERR_CODE=SwInvalidConfig plus INTR_STATE.hmac_err?

Verified: No spec text defines what a blocked start does beyond the error. Other docs only repeat that start is blocked and ERR_CODE reports SwInvalidConfig. The details of the abort case (digest cleared, FIFO cleared, no done) are specified only for a key revoked mid-operation, not for a blocked start. Nothing covers MSG_FIFO writes, MSG_LENGTH, a following hash_process, or DIGEST contents after a blocked start.

Further evidence found by the verifier:

- `hw/ip/hmac/doc/theory_of_operation.md:154`: `Triggering \`CMD.hash_start\` or \`CMD.hash_continue\` while the key is invalid is blocked, and [\`ERR_CODE\`](registers.md#err_code) reports \`SwInvalidConfig\`.` — Same vague wording; no state details.
- `hw/ip/hmac/doc/programmers_guide.md:123`: `Starting while it is still invalid is blocked with the same error.` — Again no behavior beyond the error.

Searched:

- SwInvalidConfig / invalid key / blocked wording in `hw/ip/hmac docs and hjson`: Only the same 'blocked + error' statements in the programmer's guide, theory of operation, registers.md Key_None, and the error table.
- error and recovery text in the programmer's guide in `hw/ip/hmac/doc/programmers_guide.md`: Recovery steps cover the abort case only; nothing on MSG_FIFO, process or DIGEST after a blocked start.

Looked at and ruled out:

- `hw/ip/hmac/doc/theory_of_operation.md:155-157`: Describes the key revoked during an operation, not a blocked start.

1.10 When a sideloaded key is used, CMD.hash_stop and CMD.hash_continue are rejected (saving and restoring the context is not available).

`f_099bc6608b8a` · missing detail · severity medium · section: hw/ip/hmac/doc/theory_of_operation.md > Theory of Operation > Design Details > Key sideloading > Restrictions with a sideloaded key

Gap: It is not stated what 'rejected' means for the command (ignored, state unchanged, etc.).

Spec says:

- `hw/ip/hmac/doc/programmers_guide.md:102`: `Saving and restoring the context is not available`
- `hw/ip/hmac/doc/programmers_guide.md:102`: `CMD.hash_stop and CMD.hash_continue are rejected`
- `hw/ip/hmac/doc/theory_of_operation.md:164`: `saving and restoring the context is not available with a sideloaded key: \`CMD.hash_stop\` and \`CMD.hash_continue\` are rejected`

Not answered by the spec: When CMD.hash_stop or CMD.hash_continue is rejected with a sideloaded key, does the in-progress operation continue unaffected and produce the correct final digest after hash_process? Or is it aborted, and if so what state do the FSM, DIGEST, MSG_LENGTH and the done interrupt end up in? Is the rejected command simply dropped with no other side effect besides ERR_CODE=SwInvalidConfig and the hmac_err interrupt?

Verified: No other spec text defines what "rejected" means for hash_stop/hash_continue with a sideloaded key. The docs only say the commands are rejected and ERR_CODE reports SwInvalidConfig. The key-invalid case is spelled out in detail, including the abort, the cleared state and no hmac_done. Nothing equivalent exists for rejected stop/continue.

Searched:

- hash_stop/hash_continue in hmac docs and hjson in `hw/ip/hmac (md, hjson)`: Only the theory_of_operation and programmers_guide lines (plus the generic hjson/registers.md CMD descriptions) mention these; none defines the reject behaviour.

Looked at and ruled out:

- `hw/ip/hmac/doc/theory_of_operation.md:152-157`: Describes blocked start/continue with an invalid key and the abort on key invalidation, not the rejection of stop/continue with a valid sideloaded key.

1.11 Revocation of the key by the key manager takes effect immediately.

`f_b7de457c4c27` · missing detail · severity low · section: hw/ip/hmac/doc/theory_of_operation.md > Theory of Operation > Design Details > Key sideloading

Gap: "Immediately" gives no bound on how many cycles it takes for revocation to affect the HMAC core.

Spec says:

- `hw/ip/hmac/doc/theory_of_operation.md:139`: `the key manager revoking the key takes effect immediately`

Not answered by the spec: When the key manager deasserts the sideload valid indication during an operation, within how many clock cycles must the HMAC core abort, set ERR_CODE to SwInvalidConfig, clear the SHA-2 state, digest, packer and FIFO, and return to idle? Is any processing (e.g. a block compression in flight) allowed to continue with the old key after valid drops?

Verified: Other spec text lists what happens on revocation mid-operation: abort, SwInvalidConfig, SHA-2/digest/packer/FIFO cleared, return to idle, and no done interrupt. No spec text gives a cycle bound. None says whether an in-flight block compression may continue with the old key. The ambiguity stands.

Searched:

- revoke, sideload, key valid, SwInvalidConfig in hmac md docs in `hw/ip/hmac`: Behavioural description in theory_of_operation.md, programmers_guide.md and registers.md. No cycle bound and nothing on in-flight processing.
- revoke, key_valid, SwInvalidConfig in hmac/data in `hw/ip/hmac/data`: No matches.

Looked at and ruled out:

- `hw/ip/hmac/doc/theory_of_operation.md:152-157`: Lists the reaction steps but gives no timing and does not say whether a compression in flight may finish.

---

2. Spec contradicts itself

2.1 For SHA-2 384 the digest is read from DIGEST_0-DIGEST_12.

`f_f7e6f870b119` · description vs. definition · severity high

Gap: A 384-bit digest is twelve 32-bit words (DIGEST_0-DIGEST_11), but the text gives DIGEST_0-DIGEST_12, which is thirteen registers.

Spec says:

- `hw/ip/hmac/README.md:42`: `or from DIGEST_0-DIGEST_12 for SHA-2 384`

Not answered by the spec: Is the SHA-2 384 digest read from DIGEST_0-DIGEST_11 or DIGEST_0-DIGEST_12?

Verified: The README gives DIGEST_0-DIGEST_12 for SHA-2 384. A 384-bit digest is twelve 32-bit words, so the range should be DIGEST_0-DIGEST_11. The same sentence uses DIGEST_0-DIGEST_7 for 256 bits (8 words) and DIGEST_0-DIGEST_15 for 512 bits (16 words), so the 384 case is the odd one out. I did not read the RTL or the generated register code. I did check the hjson, which defines a DIGEST register array and says nothing more specific about SHA-2 384.

Searched:

- DIGEST_0-DIGEST_N ranges in HMAC docs in `hw/ip/hmac`: Only README.md:42 says DIGEST_12. The other docs use DIGEST_0-DIGEST_15 for the full register array.
- DIGEST register definition in `hw/ip/hmac/data/hmac.hjson`: A DIGEST register array is defined, with a description referring to DIGEST_0 to DIGEST_15. No 384-specific range is given.

Same subject, also found as spec and test disagree (`f_db661743ce37`, test expects the opposite): The test randomizes the digest size, includes SHA-2 384, and reads the digest registers. The scoreboard compares them against the expected digest.

Test does:

- `hw/ip/hmac/dv/env/seq_lib/hmac_test_vectors_sha_vseq.sv:61`: `digest_size inside {SHA2_256, SHA2_384, SHA2_512};` — stimulus; If no plusarg was given, randomize the digest size among the valid values.
- `hw/ip/hmac/dv/env/seq_lib/hmac_test_vectors_sha_vseq.sv:137`: `rd_digest();` — stimulus; Read the digest registers, which triggers the scoreboard comparison.
- `hw/ip/hmac/dv/env/hmac_scoreboard.sv:496`: `` `DV_CHECK_EQ(real_digest_val, exp_digest[digest_idx]) `` — checker check; Read digest word must equal exp_digest for the expected digest size (SHA-2 256 idx<8, 384 idx<12, 512 all) when not stopped and no wipe.

Verified: The README says SHA-2 384 is read from DIGEST_0-DIGEST_12, which is 13 words. The scoreboard checks only digest_idx < 12, which is DIGEST_0-DIGEST_11, i.e. 384 bits. The two disagree. The README is likely the side that is wrong, since 384 bits is 12 words and DIGEST_12 is not part of a 384-bit digest. I did not check the RTL or the register documentation.

Further evidence found by the verifier:

- `hw/ip/hmac/README.md:41`: `or from [`DIGEST_0-DIGEST_12`](doc/registers.md#digest) for SHA-2 384` — Spec gives a range of 13 words for SHA-2 384.

---

3. Test contradicts its own description

3.1 chip_sw_hmac_enc: plan vs. test

`f_79bd89c9d1fb` · plan vs. test · severity medium

Gap: The plan says the test verifies both the HMAC done and FIFO empty interrupts. The test only checks that the serviced IRQ is HMAC done, and only for the SHA256 key-hashing step (k3). It never checks a FIFO empty interrupt. The final HMAC operation over the reference data is completed by polling, so no interrupt is verified for it.

Labels in this finding: k3 = `sw/device/tests/hmac_enc_test.c:98`

Test description says:

- `hw/top_earlgrey/data/ip/chip_hmac_testplan.hjson:13-14`: `Verify the HMAC done and FIFO empty interrupts as a part of this test.` — HMAC done and FIFO empty interrupts are verified.

Test does:

- `sw/device/tests/hmac_enc_test.c:98`: `CHECK(irq_serviced == expected_irq);` — check; The serviced IRQ is the HMAC done IRQ.
- `sw/device/tests/hmac_enc_test.c:111`: `hmac_testutils_finish_and_check_polled(&hmac, &kHmacRefExpectedDigest)` — stimulus; Poll for completion of the HMAC operation.

Verified: The plan item chip_sw_hmac_enc is explicitly linked to this test. It says the HMAC done and FIFO empty interrupts are verified. The test enables both IRQs but only checks irq_serviced == HMAC done, once, after the key-hash step. It never checks a FIFO empty IRQ, and the final HMAC is completed by polling with no interrupt check.

Searched:

- FIFO empty IRQ checks in the test source in `sw/device/tests/hmac_enc_test.c`: The FIFO empty IRQ is enabled at lines 63-70 but never checked. The handler stores irq_serviced and only the HMAC done value is compared.
- Plan-to-test link in `hw/top_earlgrey/data/ip/chip_hmac_testplan.hjson`: The plan item lists bazel //sw/device/tests:hmac_enc_test, so the link is stated in the repository.

Looked at and ruled out:

- `sw/device/tests/hmac_enc_test.c:61-70`: This enables the FIFO empty interrupt in the HMAC IP and PLIC, but enabling it is not the same as verifying it.

3.2 hmac_multistream_functest: plan vs. test

`f_05c2b1da8b2e` · plan vs. test · severity low

Gap: Plan item p2 says each streamed operation is split into two chunks. The test instead picks a random segment count for each streaming vector with rand_testutils_gen32_range(1, 4). A vector can therefore be streamed in a single segment, or in more than two, so the fixed two-chunk save/restore scenario is not guaranteed.

Labels in this finding: p2 = `hw/top_earlgrey/data/ip/chip_hmac_testplan.hjson`

Test description says:

- `hw/top_earlgrey/data/ip/chip_hmac_testplan.hjson`: `HMAC context save/restore by streaming 6 SHA-2/HMAC operations in parallel in two chunks each.`

Test does:

- `sw/device/tests/crypto/hmac_multistream_functest.c:109`: `cur_ext_vec->segment_count = rand_testutils_gen32_range(1, 4);` — stimulus; Pick a random segment count for each streaming vector.

Verified: The plan item says each operation is streamed in two chunks. The test picks a random segment count per streaming vector, so a vector can be streamed in one segment or in more than two. Two chunks is not guaranteed. I did not read the helper that implements rand_testutils_gen32_range, so I did not confirm whether 4 is a possible value. Neither reading yields a fixed two-chunk split.

Searched:

- segment_count usage and plan link in `hmac_multistream_functest.c, chip_hmac_testplan.hjson`: Random segment count; the plan item links to this test via bazel target hmac_multistream_functest.

---

4. Spec and test disagree

No verified findings.

5. Spec not tested

5.1 Partly tested: 2 requirements in README.md > Description

`f_219a1c8bf264` · severity high

1. **Requirement** `cr_03340331c331`: The module can wipe internal variables such as the secret key, intermediate hash results, digest and message scheduling array.
    - `hw/ip/hmac/README.md:59`: `is able to wipe internal variables such as the secret key, intermediate hash results, digest and the internal message scheduling array.`

    Tested by `hmac_wipe_secret`**:** Randomly triggers wipe_secret during hashing sequences. The scoreboard checker verifies that the digest read after a wipe differs from the previously expected digest.
    Not tested: Only the digest corruption is observed. Wiping of the secret key, intermediate hash state and message schedule array is not checked, nor is the digest content after the wipe.

    Tested by `hmac_secure_wipe_test`**:** Wipes with a known value after a keyed HMAC and checks every digest word equals the wipe value. A key-less restart then produces a digest different from the keyed one.
    Not tested: Wiping of the secret key, intermediate hash state and message schedule array is not directly checked; only the digest registers and an indirect result are observed.

Verified: Only digest corruption or digest values are observed. The key, hash state and schedule are not checked.

- `hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:322-327`: `task hmac_base_vseq::wipe_secrets();` — The wipe stimulus writes a random value to WIPE_SECRET, and the only check is that the digest is corrupted.
- `hw/ip/hmac/dv/env/hmac_scoreboard.sv:482-484`: `// If wipe_secret is triggered, ensure the predicted value does not match the read out` — The check is only on the digest. Internal key, hash state and message schedule are not probed.

2. **Requirement** `cr_aab3a12796d4`: Internal variables and the secret key are reset to the value written to WIPE_SECRET.

   - `hw/ip/hmac/README.md:62`: `The internal variables and secret key will be reset to the written value.`

   Tested by `hmac_secure_wipe_test`**:** Wipes with UINT32_MAX and checks that every digest word equals the wipe value. It then recomputes without a key and checks that the digest differs from the expected one.
   Not tested: Only the digest registers are observed, not the secret key, intermediate hash state or message schedule. Only one wipe value is used.

   Verified: No test checks that the internal state or key equals the wipe value, and only the digest is observed.

Items 2, 4 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- wipe_secret / secure_wipe in the hmac DV directory in `hw/ip/hmac/dv`: Only the wipe vseq and the scoreboard digest-mismatch check. No probing of internal key, state or message schedule.
- key randomization and key_length constraint and coverage in `hw/ip/hmac/dv/env`: The key array is fully randomized, the scoreboard uses only the relevant slice, key_length is randomized over all lengths, and cov bins exist.
- secure_wipe tests in sw/device/tests in `sw/device/tests`: Only BUILD files matched. The finding cites a hmac_secure_wipe_test that checks digest registers only.

Looked at and ruled out:

- `hw/ip/hmac/dv/env/hmac_env_cov.sv:84-87`: Coverage crosses key length and digest size. It does not make the wipe items any more tested.

5.2 Partly tested: 5 requirements in hmac.hjson > CFG (and 2 more sections)

`f_0e369dcd1af6` · severity high

1. **Requirement** `cr_db590d70ed32`: With Key_None and unkeyed SHA-2 (hmac_en == 0), starting is not blocked.
   Section: hw/ip/hmac/data/hmac.hjson > hmac > CFG

   - `hw/ip/hmac/data/hmac.hjson:355`: `If only unkeyed SHA-2 is configured (hmac_en == 0), starting is not blocked, since this does not require a key.`

Tested by `hmac_test_vectors_sha_vseq`**:** Runs SHA-2-only vectors (non-HMAC mode) with a key length derived per vector. The digest is compared by the scoreboard, so a blocked start would be caught.
Not tested: It is not explicit that key_length is Key_None in the unkeyed runs, nor that hmac_en == 0 with Key_None is checked as 'not blocked'.

Verified: No explicit check that hmac_en=0 with Key_None is not blocked.

- `hw/ip/hmac/dv/env/hmac_scoreboard.sv:379-387`: `if (csr_name == "status") begin` — STATUS reads are checked against a predicted idle/fifo_full value. This is generic and does not target the specific idle conditions.

2. **Requirement** `cr_1089c5d4e1c7`: While the message FIFO is full, writes to it are blocked until the FIFO has space, causing back-pressure on the interconnect.
Section: hw/ip/hmac/data/hmac.hjson > hmac > STATUS

   - `hw/ip/hmac/data/hmac.hjson:429`: `Data written to the FIFO whilst it is full will cause back-pressure on the interconnect`
   - `hw/ip/hmac/doc/programmers_guide.md:207`: `Whilst the FIFO is full, the HMAC will block writes until the FIFO has space which will cause back-pressure on the interconnect.`

   Tested by `hmac_back_pressure`**:** The test runs the smoke flow with large messages, full-width writes and back pressure enabled. The scoreboard's digest comparison would catch a write lost while the FIFO was full.
   Not tested: Nothing checks that the FIFO actually reached full or that writes are stalled until space is available. The back pressure enabled may be at the bus-agent level rather than FIFO-full blocking.

   Verified: Nothing checks that writes are stalled while the FIFO is full.

3. **Requirement** `cr_c486e3895381`: hmac_idle indicates HMAC idle status.
Section: hw/ip/hmac/data/hmac.hjson > hmac > STATUS

   - `hw/ip/hmac/data/hmac.hjson:417`: `HMAC idle status.`

   Tested by `hmac_test_vectors_hmac_vseq`**:** The HMAC test-vector sequence runs full hash operations while the scoreboard checkers compare the idle pin against the expected value after reset, after start, and at done.
   Not tested: No explicit read of STATUS.hmac_idle is shown, and the exact idle conditions are not exercised, for example the invalid-configuration path.

   Tested by `hmac_directed`**:** The directed smoke sequence runs hashes, with a reset applied at a pad FSM state, while the scoreboard idle checks compare the idle value against the expected one.
   Not tested: STATUS.hmac_idle reads are not directly shown, and the idle conditions are not specifically targeted beyond the reset case.

   Tested by `hmac_burst_wr`**:** Long-message burst-write hashing runs with the scoreboard idle checks, which compare the idle value after start and at done.
   Not tested: No direct STATUS.hmac_idle read is shown, and the idle-conditions definition is not specifically verified.

   Verified: Only the generic STATUS read prediction covers idle. The specific idle conditions are not targeted.

4. **Requirement** `cr_b0906404bd67`: The clear secret operation overwrites the internal variables with the provided 32-bit value.
   Section: hw/ip/hmac/data/hmac.hjson > hmac > WIPE_SECRET

   - `hw/ip/hmac/data/hmac.hjson:456`: `The clear secret operation overwrites the internal variables with the provided 32-bit value.`

   Tested by `hmac_wipe_secret`**:** The test runs smoke-derived sequences with randomly triggered wipe_secret. The scoreboard checker reads the digest after a wipe and checks that it differs from the previously expected one.
   Not tested: The test does not check that internal variables equal the written 32-bit value. It only checks that the digest differs. The key, intermediate state and 64-bit duplication are not verified.

   Tested by `hmac_secure_wipe_test`**:** The test writes WIPE_SECRET with UINT32_MAX and checks that every digest word equals the wipe value. It also checks that a later keyless HMAC result differs from the keyed one.
   Not tested: Only the digest registers are checked, with one wipe value and 32-bit words. The key, intermediate hash state and message schedule array are not checked, and neither is the duplicated 64-bit value for SHA-2 384/512.

   Verified: The wipe check only observes that the digest differs.

   - `hw/ip/hmac/dv/env/hmac_scoreboard.sv:482-484`: `if (cfg.wipe_secret_triggered) begin` — The wipe check is only that the digest differs. Internal key, state and schedule are not checked.
   - `hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:322-326`: `task hmac_base_vseq::wipe_secrets();` — A random wipe write is issued, with no check of internal state.

5. **Requirement** `cr_b8debfc7325d`: A software/CPU write of a 32-bit value to WIPE_SECRET wipes the internal variables, including the secret key, intermediate hash results, digest and message scheduling array.
   Section: hw/ip/hmac/data/hmac.hjson > hmac > WIPE_SECRET

   - `hw/ip/hmac/README.md:61`: `The software can wipe the internal variables and secret key by writing a 32-bit random value into WIPE_SECRET register.`
   - `hw/ip/hmac/data/hmac.hjson:455`: `If the CPU writes a value into the register, the value is used to clear some internal variables such as the secret key, intermediate hash results, digest and internal message scheduling array.`

   Tested by `hmac_wipe_secret`**:** Runs the smoke sequence with random wipe_secret writes. The scoreboard checks that the digest after the wipe differs from the expected one.
   Not tested: Only digest corruption is observed. It does not check that the secret key, intermediate hash state and message schedule are cleared, nor that the wipe value is the one written.

   Tested by `hmac_secure_wipe_test`**:** Performs a secure wipe after a keyed HMAC and checks that the digest words are overwritten. It then checks that a keyless recomputation differs from the expected digest.
   Not tested: Only the digest registers are checked directly. The wipe of the key, intermediate hash and message schedule is not directly verified.

   Verified: Internal key, state and schedule clearing are not checked.

Searched:

- Key_None with hmac_en=0 not-blocked check, item cr_db590d70ed32 in `hw/ip/hmac/dv`: Key_None is randomly exercised. The scoreboard at line 174 only covers the blocked case (Key_None with hmac_en=1). No explicit unblocked check.
- FIFO full back-pressure check, item cr_1089c5d4e1c7 in `hw/ip/hmac (fifo_full)`: The scoreboard tracks FIFO depth for status and interrupt prediction. No check that writes are stalled.
- hmac_idle checks, item cr_c486e3895381 in `hw/ip/hmac/dv`: Only the generic STATUS read prediction. No idle-condition targeted test.
- wipe_secret internal-value check, item cr_b0906404bd67 in `hw/ip/hmac/dv`: Only the digest-differs check at scoreboard line 484.
- wipe of key, state and message schedule, item cr_b8debfc7325d in `hw/ip/hmac/dv`: No check of internal key, state or schedule.

Looked at and ruled out:

- `hw/ip/hmac/dv/env/hmac_scoreboard.sv:653-720`: This tracks FIFO full and empty depth for interrupt and status prediction. It does not check that bus writes are stalled while the FIFO is full.

5.3 Partly tested: 4 requirements in hmac.hjson > KEY (and 2 more sections)

`f_0432e1ad8e9c` · severity high

2. **Requirement** `cr_67bea9a867f4`: KEY_0-KEY_31 are ignored when the key is sideloaded from the key manager.
   Section: hw/ip/hmac/data/hmac.hjson > hmac > KEY

   - `hw/ip/hmac/data/hmac.hjson:484`: `These registers are ignored when the key is sideloaded from the key manager,`
   - `hw/ip/hmac/doc/programmers_guide.md:80`: `When the sideloaded key is used, the KEY_0-KEY_31 registers and CFG.key_swap are ignored`
   - `hw/ip/hmac/README.md:45`: `in which case the KEY_0-KEY_31 registers are ignored`

   Tested by `keymgr_dpe_sideload_hmac_test`**:** The test runs HMAC with a sideloaded key and shows the digest differs from the software-key digest and matches the DV expected digest. This shows the software key was not used.
   Not tested: It does not show that KEY register writes are made and then ignored while the sideloaded key is in use.

   Verified: Sideload is disabled in block DV; no test writes KEY and then ignores it under sideload.

   - `hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:211-212`: `// TODO(#31634): Disable sideloading for now.` — Sideload is disabled in block DV, so the sideload item is untested there.

3. **Requirement** `cr_781c9e811c5e`: The KEY registers may be updated only when the engine is in Idle state.
   Section: hw/ip/hmac/data/hmac.hjson > hmac > KEY

   - `hw/ip/hmac/data/hmac.hjson:481`: `The registers are allowed to be updated only when the engine is in Idle state.`

Tested by `hmac_error_conditions_test`**:** It shows that a key write during an operation is rejected with an error.
Not tested: Only a key write while a hash is in process is checked; other non-idle states and successful updates when idle are not covered.

Verified: Only the mid-hash write is exercised; other non-idle states and the idle-success case are not.

- `hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:216-220` — Stimulus: random key written during the hash.

```
task hmac_base_vseq::write_discard_key();
  bit [TL_DW-1:0] rand_key_value = $urandom();
  int key_idx = $urandom_range(0, 31);
  csr_wr(ral.key[key_idx], rand_key_value);
```

4. **Requirement** `cr_98ab14b5706d`: Software may write the DIGEST registers when STATUS.hmac_idle is 1.
Section: hw/ip/hmac/data/hmac.hjson > hmac > DIGEST

- `hw/ip/hmac/data/hmac.hjson:507`: `When \`STATUS.hmac_idle\` is 1, these registers may be written to by software.`

Tested by `sar_same_context`**:** The sequence disables SHA and writes the saved digest back into the DIGEST registers, then resumes hashing. The scoreboard updates its expected digest on digest writes.
Not tested: The write happens after a done event with SHA disabled, not explicitly under STATUS.hmac_idle=1. The test shows no direct check that the write takes effect. It writes back the same value it read, so a dropped write might go unnoticed.

Verified: No check that a DIGEST write under hmac_idle takes effect; writes are only the same value read back.

- `hw/ip/hmac/dv/env/hmac_scoreboard.sv:338-340` — Scoreboard keys on sha_en, not STATUS.hmac_idle.

```
"digest_14", "digest_15", "msg_length_upper", "msg_length_lower": begin
          // Predict updated value coming from write iff SHA core is disabled.
          do_predict = !sha_en;
```

5. **Requirement** `cr_ce024c03a454`: Software may write MSG_LENGTH_UPPER when STATUS.hmac_idle is 1.
Section: hw/ip/hmac/data/hmac.hjson > hmac > MSG_LENGTH_UPPER

- `hw/ip/hmac/data/hmac.hjson:545`: `When \`STATUS.hmac_idle\` is 1, this register may be written by software.`

Tested by `sar_different_context`**:** Writes the message length registers while SHA is disabled and reads them back after resuming.
Not tested: Does not explicitly establish STATUS.hmac_idle=1 at the write, and only one branch checks the written value.

Verified: No explicit hmac_idle=1 check before the MSG_LENGTH_UPPER write.

Item 1 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- key writes during hash and discard checks in `hw/ip/hmac/dv`: write_discard_key, scoreboard drops key write; digest compared with model
- sideload in DV in `hw/ip/hmac/dv/env/seq_lib`: sideload forced to 0 with a TODO
- hmac_idle use around digest/msg_length writes in `hw/ip/hmac/dv`: scoreboard keys on sha_en; no explicit hmac_idle check

Looked at and ruled out:

- `hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:392-400`: Key write during hash is covered, but only in the hashing state, not other non-idle states.

5.4 Partly tested: 1 requirement in hmac.hjson > MSG_FIFO

`f_248760d86493` · severity high

1. **Requirement** `cr_5dc8e5c046fb`: For writes within the MSG_FIFO window, only address bits [1:0] are used to determine the write offset (to handle non-32-bit writes).

   - `hw/ip/hmac/data/hmac.hjson:565-566`: `Only the lower [1:0] bits of the address matter to writes within the window (for correctly dealing with non 32-bit writes)`

   Tested by `hmac_smoke`**:** Messages are written to the window as bursts or single writes, and the digest is checked end to end. Nothing shown ensures sub-word or unaligned writes that exercise address bits [1:0].
   Not tested: Writes of sub-word size or with non-zero address bits [1:0], and a check that the offset is determined only by those bits.

   Tested by `hmac_burst_wr`**:** Burst writes to the message FIFO are checked end to end through the digest comparison. No explicit sub-word or unaligned write offset is exercised.
   Not tested: Writes of sub-word size or with non-zero address bits [1:0], and a check that the offset is determined only by those bits.

   Verified: Sub-word writes exist, but the scoreboard ignores the address and the mask is independent of address bits [1:0]. Nothing checks that the offset depends only on those bits.

   - `hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:358`: `` `DV_CHECK_FATAL(randomize(wr_addr, wr_mask) with {$countones(wr_mask) <= msg_q.size();}) `` — Stimulus: random in-window address and a random byte mask. Address and mask are chosen independently, so the mask is not derived from address bits [1:0].
   - `hw/ip/hmac/dv/env/hmac_scoreboard.sv:157-162`: `foreach (item.a_mask[i]) begin` — The scoreboard builds the expected message from a_mask only and ignores the address, so it can't detect a wrong address-[1:0]-to-offset mapping.
   - `hw/dv/sv/tl_agent/tl_seq_item.sv:37`: `` `define ADDR_ALIGNED_TO_SIZE(a_addr, a_size) ((a_addr & ((1 << a_size) - 1)) == 0) `` — TL requires natural alignment, which limits how address bits [1:0] can be combined with size and mask.

Searched:

- byte-lane or address [1:0] handling in the HMAC dv in `hw/ip/hmac/dv`: Only the random wr_addr/wr_mask in hmac_base_vseq, and the mask-based scoreboard model. No check on address bits [1:0].
- how tl_access handles the address low bits in `hw/dv/sv/tl_agent`: Only alignment constraints in tl_seq_item. I did not fully trace how tl_access sets the size.

5.5 Partly tested: 3 requirements in programmers_guide.md > Using a sideloaded key (and 1 more section)

`f_5a3999906c5e` · severity high

1. **Requirement** `cr_59caa8cb74a9`: Starting while the sideloaded key is still invalid is blocked with the same error.
   Section: hw/ip/hmac/doc/programmers_guide.md > Programmer's Guide > Using a sideloaded key

   - `hw/ip/hmac/doc/programmers_guide.md:123`: `Starting while it is still invalid is blocked with the same error.`

   Tested by `keymgr_dpe_sideload_hmac_test`**:** The test starts HMAC with an invalid sideload key and checks that SwInvalidConfig is reported and the operation does not complete.
   Not tested: The recovery scenario is not exercised: the start is not made after an earlier abort, with hmac_err cleared and the key still invalid.

   Verified: No test restarts with hmac_err cleared and the key still invalid. The IP-level DV has no sideload stimulus.

2. **Requirement** `cr_a578aba72de1`: These errors are signalled through INTR_STATE.hmac_err.
   Section: hw/ip/hmac/doc/programmers_guide.md > Programmer's Guide > Using a sideloaded key

   - `hw/ip/hmac/doc/programmers_guide.md:112`: `In all of these cases the error is signalled through INTR_STATE.hmac_err, as for any other HMAC error.`

   Tested by `keymgr_dpe_sideload_hmac_test`**:** The test checks that hmac_err is set for the rejected hash_stop and for the start with an invalid sideload key.
   Not tested: The abort when the key is revoked during a running operation is not exercised, so the hmac_err signalling is not checked for that case.

   Verified: No test revokes the key during a running operation and checks hmac_err.

3. **Requirement** `cr_fb14b2d5a9e6`: The CFG.key_length read-back value does not depend on the key manager providing a key and is available before an operation starts.
   Section: hw/ip/hmac/doc/programmers_guide.md > Programmer's Guide > Using a sideloaded key

   - `hw/ip/hmac/doc/programmers_guide.md:82`: `so the value read back does not depend on the key manager providing a key and is available before an operation is started`

   Tested by `keymgr_dpe_sideload_hmac_test`**:** The test writes CFG with sideload and hmac_en set and checks that CFG.key_length reads back as KEY_512, the sideload interface width. This only happens in the run where the key manager has generated a valid key.
   Not tested: No check that the read-back value is the same while the key manager provides no valid key, for example after the key is cleared. It is also not clear that CFG is read before an operation starts.

Verified: No test checks that CFG.key_length reads back the same with no valid key from the key manager.

Item 4 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- sideload in hmac DV in `hw/ip/hmac/dv`: Only tb, env config, package and base vseq plumbing. The scoreboard has no sideload handling and there is no sideload vseq.
- SwInvalidConfig / Key_None in `hw/ip/hmac/dv`: The scoreboard predicts SwInvalidConfig on an invalid config, and the smoke vseq randomizes key_length including illegal values.

Looked at and ruled out:

- `hw/ip/hmac/dv/env/seq_lib/hmac_smoke_vseq.sv:178`: Handles invalid_cfg but has no sideload or key-revoke scenario, so it does not cover items 1-3.

5.6 Partly tested: 4 requirements in theory_of_operation.md > Key sideloading (and 2 more sections)

`f_5ef6f110522a` · severity high

1. **Requirement** `cr_b5bc42e67d01`: Triggering CMD.hash_start or CMD.hash_continue while the sideloaded key is invalid is blocked.
   Section: hw/ip/hmac/doc/theory_of_operation.md > Theory of Operation > Design Details > Key sideloading

   - `hw/ip/hmac/doc/theory_of_operation.md:154`: `Triggering CMD.hash_start or CMD.hash_continue while the key is invalid is blocked`

   Tested by `keymgr_dpe_sideload_hmac_test`**:** Starts HMAC with an invalid sideload key (key cleared, no new randomness loaded) and checks that hmac_err is raised with SwInvalidConfig and hmac_done stays unset, so the start is blocked.
   Not tested: Only CMD.hash_start is exercised with an invalid key; CMD.hash_continue with an invalid key is not tested.

   Verified: No test issues hash_continue with an invalid sideload key.

   - `hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:211-212`: `// TODO(#31634): Disable sideloading for now.` — Block-level DV never enables sideload.

2. **Requirement** `cr_d468d703ae52`: The sideloaded key length overrides the length configured in CFG.key_length.
   Section: hw/ip/hmac/doc/theory_of_operation.md > Theory of Operation > Design Details > Key sideloading

   - `hw/ip/hmac/doc/theory_of_operation.md:141`: `so it overrides whatever is configured in CFG.key_length.`

   Tested by `keymgr_dpe_sideload_hmac_test`**:** Runs HMAC with the sideloaded key and checks that CFG.key_length reads back as KEY_512, the sideload interface width, which shows the sideload length is in use.

Not tested: The test does not show that a different CFG.key_length was configured before the sideloaded run, so the override of a conflicting configured length is not clearly verified.

Verified: No test configures a conflicting CFG.key_length before the sideloaded run.

- `sw/device/tests/keymgr_dpe_sideload_hmac_test.c:122`: `* - saving the context with `CMD.hash_stop` is rejected and has no effect.` — Only hash_stop is tested with sideload.

3. **Requirement** `cr_6bc711c2b717`: ERR_CODE reports SwInvalidConfig when hash_stop or hash_continue is rejected with a sideloaded key.
   Section: hw/ip/hmac/doc/theory_of_operation.md > Theory of Operation > Design Details > Key sideloading > Restrictions with a sideloaded key

   - `hw/ip/hmac/doc/theory_of_operation.md:164`: `[`ERR_CODE`](registers.md#err_code) reports `SwInvalidConfig``

   Tested by `keymgr_dpe_sideload_hmac_test`**:** During a sideloaded-key operation the test writes CMD.hash_stop. It then checks that the hmac_err interrupt is set and ERR_CODE equals SwInvalidConfig.
   Not tested: Only hash_stop is exercised. hash_continue with a sideloaded key is never issued, so its SwInvalidConfig reporting is not verified.

   Verified: Only hash_stop is exercised with a sideloaded key.

4. **Requirement** `cr_f3333982706d`: When a sideloaded key is used, CMD.hash_stop and CMD.hash_continue are rejected (saving and restoring the context is not available).
   Section: hw/ip/hmac/doc/theory_of_operation.md > Theory of Operation > Design Details > Key sideloading > Restrictions with a sideloaded key

   - `hw/ip/hmac/doc/programmers_guide.md:102`: `Saving and restoring the context is not available`
   - `hw/ip/hmac/doc/programmers_guide.md:102`: `CMD.hash_stop and CMD.hash_continue are rejected`
   - `hw/ip/hmac/doc/theory_of_operation.md:164`: `saving and restoring the context is not available with a sideloaded key: `CMD.hash_stop` and `CMD.hash_continue` are rejected`

   Tested by `keymgr_dpe_sideload_hmac_test`**:** The test attempts CMD.hash_stop during a sideloaded-key operation. It checks that the hmac_err interrupt is set and ERR_CODE is SwInvalidConfig, which shows the command is rejected.
   Not tested: CMD.hash_continue with a sideloaded key is not tested. The test also does not check that the HMAC state is otherwise unchanged after the rejection.

   Verified: hash_continue is never issued with a sideloaded key.

Item 5 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- hash_continue/SwInvalidConfig in hmac dv and top_earlgrey dv in `hw/ip/hmac/dv`, `hw/top_earlgrey/dv`: Only scoreboard non-sideload handling; no sideload continue tests
- sideload in hmac dv and sw tests in `repo`: Disabled in block DV; only keymgr_dpe_sideload_hmac_test
- wr_mask in hmac_base_vseq in `hw/ip/hmac/dv`: random contiguous masks

Looked at and ruled out:

- `hw/ip/hmac/dv/env/hmac_scoreboard.sv:198-222`: SwInvalidConfig modeling is for Key_1024 with SHA2_256, not the sideload case.

5.7 Partly tested: 4 requirements in theory_of_operation.md > SHA-2 message feed and pad (and 2 more sections)

`f_01ccd4ae6baa` · severity high

1. **Requirement** `cr_26f0103d3cea`: When the FIFO is full, writes are blocked, causing interconnect back-pressure rather than being dropped or overwriting FIFO contents.
   Section: hw/ip/hmac/doc/theory_of_operation.md > Theory of Operation > Design Details > SHA-2 message feed and pad

   - `hw/ip/hmac/doc/theory_of_operation.md:34`: `If the FIFO is full, the HMAC block will block any writes leading to back-pressure on the interconnect (as opposed to dropping those writes or overwriting existing FIFO contents).`

   Tested by `hmac_back_pressure`**:** Heavy full-width writes with back pressure enabled, with the digest checked by the scoreboard. Dropped or overwritten writes would corrupt the digest.
   Not tested: Does not guarantee the FIFO becomes full, and does not check that interconnect back-pressure is applied.

   Verified: No check that the FIFO becomes full or that back-pressure is applied. The scoreboard only tracks fifo_full status.

2. **Requirement** `cr_0f006fc77258`: An aborted operation does not raise the hmac_done interrupt.
   Section: hw/ip/hmac/doc/theory_of_operation.md > Theory of Operation > Design Details > Key sideloading

   - `hw/ip/hmac/doc/programmers_guide.md:113`: `An aborted operation does not raise the hmac_done interrupt`
   - `hw/ip/hmac/doc/theory_of_operation.md:157`: `No \`hmac_done\` interrupt is raised for an aborted operation.`

   Tested by `keymgr_dpe_sideload_hmac_test`**:** The test starts HMAC with an invalid sideload key and checks that hmac_err is raised with SwInvalidConfig and that the hmac_done interrupt is not set.
   Not tested: It does not abort an operation that is already running by revoking the key mid-operation, so the no-done behavior of an aborted in-flight operation is not verified.

   Verified: No HMAC test aborts an in-flight operation and checks that hmac_done stays clear.

3. **Requirement** `cr_605b3802525f`: Revocation of the key by the key manager takes effect immediately.
   Section: hw/ip/hmac/doc/theory_of_operation.md > Theory of Operation > Design Details > Key sideloading

   - `hw/ip/hmac/doc/theory_of_operation.md:139`: `the key manager revoking the key takes effect immediately`

   Tested by `keymgr_dpe_sideload_hmac_test`**:** After the sideload key is cleared, the next HMAC start is blocked with SwInvalidConfig. This shows a revoked key is seen by HMAC, but only for an operation

started afterwards.
Not tested: Revocation during an operation in progress, and any timing or immediacy of the effect, are not checked.

Verified: No test revokes the key mid-operation or measures how quickly the revocation takes effect.

4. **Requirement** `cr_75181956206e`: ERR_CODE reports SwInvalidConfig when hash_start or hash_continue is triggered while the key is invalid.
Section: hw/ip/hmac/doc/theory_of_operation.md > Theory of Operation > Design Details > Key sideloading

   - `hw/ip/hmac/doc/theory_of_operation.md:154`: `ERR_CODE reports SwInvalidConfig.`

   Tested by `keymgr_dpe_sideload_hmac_test`**:** The test clears the sideloaded key, starts HMAC with the invalid key, and checks that hmac_err is raised with ERR_CODE SwInvalidConfig and that hmac_done is not set.
   Not tested: Only the CMD.hash_start path is exercised; triggering CMD.hash_continue while the key is invalid is not checked.

   Verified: The only sideload-with-invalid-key test I found is the chip-level keymgr sideload test the finding cites, which exercises hash_start. I found no check of hash_continue with an invalid key.

Item 2 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- FIFO-full / back-pressure checks in `hw/ip/hmac/dv`: The scoreboard only tracks fifo_full status and detection. I found no check that a TL response is delayed while the FIFO is full.
- SwInvalidConfig and Key_1024 handling in `hw/ip/hmac/dv`: The scoreboard predicts SwInvalidConfig on hash start. The test-vector sequence skips invalid vectors.
- SwInvalidConfig in the chip-level keymgr sideload vseq in `hw/top_earlgrey/dv/env/seq_lib/chip_sw_keymgr_dpe_sideload_hmac_vseq.sv`: No matches

Looked at and ruled out:

- `hw/ip/hmac/dv/env/seq_lib/hmac_test_vectors_sha_vseq.sv:97-105`: Invalid key-length and digest combinations are skipped here, so this sequence does not test rejection.

5.8 Test plan items with no test: 2 plan items

`f_d4135d06ef2a` · severity medium

2. **Plan item:** TB_LINT_COMPLETE — Testbench lint is complete, with CI lint reporting no errors or warnings and no waivers. (`hw/ip/hmac/doc/checklist.md`)

   Verified: This is a lint checklist item with no test. I did not check whether the CI lint results back the 'Done' status.

3. **Plan item:** tl_intg_err_cg — Coverage group for kinds of integrity errors (command, data, both) and number of error bits per integrity check, including byte-enabled writes on memories that store integrity values. (`hw/dv/tools/dvsim/testplans/tl_device_access_types_testplan.hjson`)

Verified: It is a covergroup with no test mapped to it. I did not look for a separate TL integrity error test, so this is not a firm finding.

- `hw/dv/tools/dvsim/testplans/tl_device_access_types_testplan.hjson:21`: `name: tl_intg_err_cg` — This is a covergroup, and no test is attached to it.

Items 1, 3, 5 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- HMAC imports of common test and testplan hjsons in `hw/ip/hmac/dv/hmac_sim_cfg.hjson, hw/ip/hmac/data/hmac_testplan.hjson`: csr_tests and alert_test hjsons are imported, and the common testplans are imported.
- regwen testpoint test list in `csr_testplan.hjson`: It maps to csr_rw and csr_aliasing.

5.9 Test plan items with no test: 3 plan items

`f_f08983440cee` · severity medium

1. **Plan item:** SIM_CODE_COVERAGE_AT_100 — Simulation code coverage reaches 100%. (`hw/ip/hmac/doc/checklist.md`)

   Verified: A coverage-closure goal. No test implements it. The checklist cites an issue instead.

   - `hw/ip/hmac/doc/checklist.md:247-249`: `Coverage | [SIM_CODE_COVERAGE_AT_100][] | Done | Documented in issue #25216` — The coverage checklist items are process sign-offs, not tests. FPV is N/A.

2. **Plan item:** SIM_FUNCTIONAL_COVERAGE_AT_100 — Simulation functional coverage reaches 100%. (`hw/ip/hmac/doc/checklist.md`)

   Verified: A coverage-closure goal. No test implements it.

3. **Plan item:** FPV_CODE_COVERAGE_AT_100 — FPV code coverage reaches 100%; marked N/A for HMAC. (`hw/ip/hmac/doc/checklist.md`)

   Verified: The item is marked N/A for HMAC, so it has no test.

Items 2, 4 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- fifo_empty_status_interrupt and csr_aliasing across hmac and hw/dv in `hw/ip/hmac, hw/dv`: The testplan lists tests for fifo_empty. The csr_aliasing test is generated from csr_tests.hjson.
- checklist coverage items in `hw/ip/hmac/doc/checklist.md`: SIM code and functional coverage at 100% are Done with issue references. FPV code coverage is N/A.

5.10 Test plan items with no test: 3 plan items

`f_f1c31aadbabb` · severity medium

1. **Plan item:** X_PROP_ANALYSIS_COMPLETED — X-propagation analysis is completed, evidenced by the xprop log reporting full instrumentation success. (`hw/ip/hmac/doc/checklist.md`)

Verified: The only evidence cited is the xprop log; no test implements it.

- `hw/ip/hmac/doc/checklist.md:244`: `xprop.log file says: "instrumentation success rate: 100%"` — X-prop item is evidenced by a tool log, not a test.

2. **Plan item:** ALL_TODOS_RESOLVED — All TODOs resolved; remaining TODOs have linked issues and are not needed for verification completion. (`hw/ip/hmac/doc/checklist.md`)

   Verified: A process item with no test.

   - `hw/ip/hmac/doc/checklist.md:251`: `Remaining TODOs have a linked issue and are not required for verification completion` — A process item with no test.

3. **Plan item:** PRE_VERIFIED_SUB_MODULES_V3 — Pre-verified sub-modules at V3; marked N/A. (`hw/ip/hmac/doc/checklist.md`)

   Verified: Marked N/A, so no test is expected.

   - `hw/ip/hmac/doc/checklist.md:254`: `[PRE_VERIFIED_SUB_MODULES_V3][] | N/A` — Marked N/A.

Searched:

- X_PROP_ANALYSIS_COMPLETED / xprop in `hw/ip/hmac`: Only the checklist.md row, no test.
- ALL_TODOS_RESOLVED in `hw/ip/hmac`: Only the checklist.md row, no test.
- PRE_VERIFIED_SUB_MODULES_V3 in `hw/ip/hmac`: Only the checklist.md row, marked N/A, no test.

5.11 No test found: 3 requirements in README.md > Description

`f_29488fe28d17` · severity medium

1. **Requirement** `cr_419b9981b884`: A sideloaded key is never exposed to software.

   - `hw/ip/hmac/doc/programmers_guide.md:77`: `The key is then never exposed to software.`
   - `hw/ip/hmac/README.md:45`: `the key is never exposed to software.`

   Verified: DV disables sideload, and no check shows the key is hidden from software.

   - `hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:211-212`: `// TODO(#31634): Disable sideloading for now.` — DV forces cfg.sideload to 0, so a sideloaded key is never exercised and non-exposure cannot be checked.

2. **Requirement** `cr_4282db4ce90a`: Software cannot read from the message FIFO.

   - `hw/ip/hmac/README.md:60`: `which SW writes the message to (but cannot read from).`

   Verified: I found no test or check that reads the message FIFO from software or confirms that it cannot be read.

3. **Requirement** `cr_8e41a2412a72`: Wiping does not clear the message FIFO.

   - `hw/ip/hmac/README.md:60`: `It does not wipe the message FIFO`

   Verified: The wipe_secret test only checks that the digest changed. Nothing checks that the message FIFO is left intact.

- `hw/ip/hmac/dv/env/seq_lib/hmac_wipe_secret_vseq.sv:6-7`: `// During the transaction, this sequence randomly triggers wipe_secret and uses scb to check if` — The wipe check only covers the digest registers. Nothing checks that the message FIFO is left intact.

Item 3 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- sideload, key_valid and keymgr in the HMAC DV directory in `hw/ip/hmac/dv`: The sideload agent is wired in, but sideload is disabled in the base vseq. No check that the key is hidden from software.
- message FIFO read or wipe behaviour in the testplan and scoreboard in `hw/ip/hmac/data/hmac_testplan.hjson and the scoreboard`: Nothing checks that software cannot read the FIFO, and nothing checks that wipe leaves the FIFO intact.
- DIGEST readout for SHA-384 in `hw/ip/hmac/dv/env/hmac_scoreboard.sv`: The scoreboard compares digest words 0-11 for SHA-384.
- sideload and wipe_secret in HMAC software tests in `sw/device/tests`: Found hmac_secure_wipe_test.c and keymgr_dpe_sideload_hmac_test.c, but I did not read them, so this search does not show whether they check non-exposure of the key or FIFO behaviour.

Looked at and ruled out:

- `sw/device/tests/keymgr_dpe_sideload_hmac_test.c:1`: I did not read this file. It is a sideload HMAC test and probably checks only the resulting HMAC value, not that the key is hidden from software.
- `hw/ip/hmac/dv/env/seq_lib/hmac_wipe_secret_vseq.sv:38-43`: It triggers wipe_secret, but the scoreboard only checks the digest registers, not the message FIFO.

5.12 No test found: 1 requirement in hmac.hjson > CFG

`f_3fc7728836ce` · severity medium

3. **Requirement** `cr_5a80d0a89e9e`: CFG.sideload: hardware receives a write-qualifier pulse on software writes (true).

   - `hw/ip/hmac/data/hmac.hjson:203`: `hwqe: "true"`

   Verified: Sideload is explicitly disabled in the DV, so no test writes CFG.sideload.

   - `hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:211-212` — Sideload is forced off in random CFG writes, so no test exercises CFG.sideload.

     ```
     // TODO(#31634): Disable sideloading for now.
       rand_config_value[ral.cfg.sideload.get_lsb_pos()] = 1'b0;
     ```

Items 1, 2, 4, 5 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- sideload, sha_en, hmac_en and digest_swap in the HMAC DV directory in `hw/ip/hmac/dv`: Sideload is disabled in random CFG writes. The smoke vseq drives sha_en, hmac_en and digest_swap, and the scoreboard predicts the digest from them.

Looked at and ruled out:

- `hw/ip/hmac/dv/env/hmac_env.sv:32`: A sideload agent is instantiated, but it is never enabled.

5.13 No test found: 4 requirements in hmac.hjson > CFG

`f_566800bb50a7` · severity medium

1. **Requirement** `cr_01eaae6544c4`: CFG.sideload: the register is implemented outside the register block (true).
   - `hw/ip/hmac/data/hmac.hjson:202`: `hwext: "true"`

   Verified: sideload is not a field of CFG. The hwext attribute is structural and no test checks it.

2. **Requirement** `cr_030b1d53349f`: CFG.digest_swap: hardware receives a write-qualifier pulse on software writes (true).
   - `hw/ip/hmac/data/hmac.hjson:203`: `hwqe: "true"`

   Verified: No test checks the hwqe pulse for digest_swap; it is only exercised implicitly.

3. **Requirement** `cr_1046bc3a01de`: CFG.key_length: hardware receives a write-qualifier pulse on software writes (true).
   - `hw/ip/hmac/data/hmac.hjson:203`: `hwqe: "true"`

   Verified: No test checks the hwqe pulse for key_length; it is only exercised implicitly.

4. **Requirement** `cr_2d1b1f596874`: CFG.digest_size: the register is implemented outside the register block (true).
   - `hw/ip/hmac/data/hmac.hjson:202`: `hwext: "true"`

   Verified: The hwext attribute for digest_size is not checked by any test.

Item 4 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- digest_swap/key_length/digest_size/sideload in hmac dv in `hw/ip/hmac/dv`: Functional use in the scoreboard and sequences. No test checks hwext or hwqe behavior as such.
- digest clear on sha_en disable in `hw/ip/hmac/dv`: Scoreboard prediction at lines 306-309 and smoke vseq digest read at lines 256-264.

Looked at and ruled out:

- `hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:619-628`: The save-and-restore sequences disable sha_en but rewrite the digest without reading it in between, so they do not check the clear.

5.14 Partly tested: 1 requirement in hmac.hjson > CFG

`f_81c7d089aa21` · severity medium

2. **Requirement** `cr_403238c02dda`: key_length value is irrelevant when the key is sideloaded.

- `hw/ip/hmac/data/hmac.hjson:315`: `The value of this register is irrelevant when HMAC is used as a plain SHA-2 hash function without a key (i.e., CFG.hmac_en is 0), and when the key is sideloaded from the key manager (see CFG.sideload).`

Tested by `keymgr_dpe_sideload_hmac_test`**:** Runs HMAC with a keymgr_dpe sideloaded key and checks the digest, and the CFG key_length field reads back as KEY_512 during the sideloaded run. This shows hardware overriding the key length for sideload.
Not tested: Does not program different key_length values under sideload to show the configured value has no effect on the digest.

Verified: Only the keymgr_dpe sideload test exists, and I found no stimulus that varies key_length under sideload.

Items 1, 3, 4, 5 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- key_length, SHA2_None and sideload in hw/ip/hmac/dv in `hw/ip/hmac/dv`: Randomized key_length and digest_size including illegal values, scoreboard prediction of the casts, and invalid-config checks. Sideload appears only in the tb and env plumbing. I found no sequence that varies key_length under sideload.

Looked at and ruled out:

- `hw/ip/hmac/dv/env/hmac_scoreboard.sv:173-177`: This covers the hash_start and process invalid-config path. It does not clearly show a distinct hash_continue trigger being blocked.

5.15 No test found: 2 requirements in hmac.hjson > CFG

`f_d274e64b1f17` · severity medium

2. **Requirement** `cr_d00d322b1664`: CFG.key_swap: hardware receives a write-qualifier pulse on software writes (true).
   - `hw/ip/hmac/data/hmac.hjson:203`: `hwqe: "true"`

   Verified: No test directly checks the hwqe pulse for key_swap.

3. **Requirement** `cr_f7398c7e530f`: CFG.endian_swap: hardware receives a write-qualifier pulse on software writes (true).
   - `hw/ip/hmac/data/hmac.hjson:203`: `hwqe: "true"`

   Verified: No test directly checks the hwqe pulse for endian_swap.

Items 1, 3, 4 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- endian_swap/key_swap/sha_en usage in `hw/ip/hmac/dv`: Used in vseqs, scoreboard and coverage. The scoreboard predicts digests with these settings.
- hwqe/write-qualifier checks on CFG in `hw/ip/hmac/dv`: No direct check. Only implicit effect through the config taking effect.

5.16 Only weak evidence (no check that can fail): 1 requirement in hmac.hjson > CFG (and 1 more section)

`f_446059c4cf4b` · severity medium

5. **Requirement** `cr_302cc8a11faa`: Writing 1 to hash_stop causes the hmac_done interrupt to be set once the current block has been hashed.
   Section: hw/ip/hmac/data/hmac.hjson > hmac > CMD

   - `hw/ip/hmac/data/hmac.hjson:396`: `When 1 is written to this field, SHA-2 or HMAC will afterwards set the `hmac_done` interrupt as soon as the current block has been hashed.`

   Nearest evidence (not counted as a test): `sar_same_context` (stimulus only, no check); `sar_stop_and_continue` (stimulus only, no check); `sar_different_context` (stimulus only, no check)

   Verified: Only a spurious hmac_done is checked, plus an implicit hang or timeout if it never arrives. The timing 'as soon as the current block has been hashed' is not checked.

   - `hw/ip/hmac/dv/env/hmac_scoreboard.sv:412-417` — For hash_stop the scoreboard only checks that hmac_done is not spurious. It does not check the timing of the done interrupt after the current block.

     ```
     // here check DUT should only trigger hmac_done when sha is enabled, and
              // previously triggered hash_process or hash_stop.
     ```

Items 1, 2, 3, 4 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- key_length and digest_size use in the DV env in `hw/ip/hmac/dv`: Random one-hot key_length and digest_size constraints in hmac_base_vseq. The scoreboard compares digests against DPI models for every key length and digest size.
- hash_stop in the scoreboard in `hmac_scoreboard.sv`: Only a comment at line 413; no explicit check of hash_stop timing.

Looked at and ruled out:

- `hw/ip/hmac/dv/env/hmac_scoreboard.sv:173-176`: This invalid-config detection is relevant to key_length and digest_size, but it does not test the valid one-hot selections.

5.17 Only weak evidence (no check that can fail): 1 requirement in hmac.hjson > CMD (and 1 more section)

`f_217a4f6359ea` · severity medium

1. **Requirement** `cr_ef5931e3a47f`: After hash_stop completes, the hash can be read from DIGEST_0 to DIGEST_15.
   Section: hw/ip/hmac/data/hmac.hjson > hmac > CMD

   - `hw/ip/hmac/data/hmac.hjson:397`: `The hash can then be read from the registers !!DIGEST_0 to !!DIGEST_15.`

   Nearest evidence (not counted as a test): `sar_same_context` (stimulus only, no check)

Verified: Digest read after stop is explicitly not compared.

- `hw/ip/hmac/dv/env/hmac_scoreboard.sv:474-481`: `// Predict intermediate digest values when S&R has been triggered` — Intermediate digest after hash_stop is not compared (TODO #23240); reads only compared when !hmac_stopped.

Item 2 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- key_length usage in DV in `hw/ip/hmac/dv`: scoreboard predict_digest selects key words by key_length; base vseq randomizes key_length
- digest checks around hash_stop in `hmac_scoreboard.sv`: comparison skipped while hmac_stopped

---

5.18 No test found: 1 requirement in hmac.hjson > DIGEST (and 2 more sections)

`f_dcd02c1afd9c` · severity medium

4. **Requirement** `cr_959451b086ac`: The lower 3 bits [2:0] of the message length are ignored.
   Section: hw/ip/hmac/data/hmac.hjson > hmac > MSG_LENGTH_LOWER

   - `hw/ip/hmac/data/hmac.hjson:529`: `Lower 3 bits [2:0] are ignored.`

   Verified: No stimulus writes nonzero values to bits [2:0]. The SAR values are multiples of 1024, and the comment says other values are unsupported.

   - `hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:700-707`: `// Override the msg_length value to be able to verify the upper part of the register.` — SAR test writes only multiples of 512/1024, so bits [2:0] are never nonzero.

Items 1, 2, 3, 5 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- msg_length usage in the hmac dv directory in `hw/ip/hmac/dv`: SAR writes and reads in hmac_base_vseq. The only explicit check is DV_CHECK_EQ at line 726, and its inputs are multiples of 512/1024.
- scoreboard handling of digest and msg_length writes in `hmac_scoreboard.sv`: Predicted on write when sha_en is off. Digest reads are checked against exp_digest.

Looked at and ruled out:

- `hw/ip/hmac/dv/env/hmac_scoreboard.sv:522-541`: Message length reads are only sampled for coverage, and the read check is disabled when sar_skip_ctxt is set. No low-bits-ignored check.

5.19 No test found: 1 requirement in hmac.hjson > STATUS (and 3 more sections)

`f_aedc0fead5b4` · severity medium

2. **Requirement** `cr_1c762e6535de`: For SHA-2 384/512 modes operating on 64-bit words, the 32-bit wipe value is replicated and concatenated to form the 64-bit wipe value.
   Section: hw/ip/hmac/data/hmac.hjson > hmac > WIPE_SECRET

- `hw/ip/hmac/README.md:63`: `For SHA-2 384/512 modes that operate on 64-bit words, the 32-bit random value is replicated and concatenated to create the 64-bit value.`
- `hw/ip/hmac/data/hmac.hjson:457`: `For SHA-2 384/512 that work with 64-bit words, the 32-bit value is duplicated and concatenated to generate the 64-bit value.`

Verified: The wipe value is never checked. Only digest != expected is asserted.

- `hw/ip/hmac/dv/env/hmac_scoreboard.sv:484-485` — After a wipe the digest is only checked to differ from the expected value. The 32-bit to 64-bit replication is not checked.

```
if (cfg.wipe_secret_triggered) begin
              `DV_CHECK_NE(real_digest_val, exp_digest[digest_idx])
```

Items 1, 3, 4, 5 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- wipe_secret checks in `hw/ip/hmac/dv`: Only a DV_CHECK_NE on the digest after a wipe. No check of the replicated 64-bit wipe value.
- idle, key and digest usage in `hw/ip/hmac/dv`: The scoreboard checks idle, the key is written in the base vseq, and the digest is compared.

5.20 No test found: 3 requirements in interfaces.md > Interrupts

`f_7a0abc79a6f5` · severity medium

1. **Requirement** `cr_8532dccc2de0`: The fifo_empty interrupt is not raised while the HMAC block is running in HMAC mode and performing the second round, i.e. computing the final hash of the outer key and the first-round result obtained with the inner key.

   - `hw/ip/hmac/data/hmac.hjson:70`: `i) The HMAC block is not running in HMAC mode and performing the second round of computing the final hash of the outer key as well as the result of the first round using the inner key.`
   - `hw/ip/hmac/doc/interfaces.md:24`: `i) The HMAC block is not running in HMAC mode and performing the second round of computing the final hash of the outer key as well as the result of the first round using the inner key.`

   Verified: The second-round exclusion exists only as a comment.

   - `hw/ip/hmac/dv/env/hmac_scoreboard.sv:683-685`: `// The FIFO empty interrupt is raised only if the message FIFO is actually writable by` — Condition 1 appears only in a comment.
   - `hw/ip/hmac/dv/env/hmac_scoreboard.sv:693`: `if (fifo_full_detected && hmac_fifo_empty) begin` — The prediction has no second-round term.

2. **Requirement** `cr_eb1d91a4dabf`: hmac_err is an Event-type interrupt.

   - `hw/ip/hmac/doc/interfaces.md:25`: `hmac_err`
   - `hw/ip/hmac/doc/interfaces.md:25`: `Event`

   Verified: No test checks that hmac_err is Event-type.

3. **Requirement** `cr_ec58bab546a9`: fifo_empty is a Status-type interrupt.

- `hw/ip/hmac/doc/interfaces.md:24`: `fifo_empty`
- `hw/ip/hmac/doc/interfaces.md:24`: `Status`

Verified: No test checks that fifo_empty is Status-type.

Searched:

- second-round / HMAC-mode exclusion of fifo_empty in `hw/ip/hmac/dv/env (scoreboard, vseqs)`: Only a comment in the scoreboard. No code term and no directed stimulus.
- hmac_err interrupt type (Event) checks in `hw/ip/hmac/dv`: Only generic intr_state prediction. No check on the interrupt type.
- fifo_empty interrupt type (Status) checks in `hw/ip/hmac/dv`: Scoreboard predicts a sticky bit from the FIFO-full-then-empty condition. No Status-type check.

Looked at and ruled out:

- `hw/ip/hmac/dv/env/hmac_scoreboard.sv:389-392`: Predicts intr_state.fifo_empty generically. It does not model the second-round exclusion or the interrupt type.

5.21 No test found: 5 requirements in interfaces.md > Hardware Interfaces (and 1 more section)

`f_78aab0dc03f7` · severity medium

1. **Requirement** `cr_179a3654a746`: The hmac module has no TL-UL bus host interfaces.
   Section: hw/ip/hmac/doc/interfaces.md > Hardware Interfaces

   - `hw/ip/hmac/doc/interfaces.md:8`: `Bus Host Interfaces (TL-UL): *none*`

   Verified: Static statement; no test checks it.

2. **Requirement** `cr_1ce0d9febabc`: The hmac module uses clk_i as its primary clock.
   Section: hw/ip/hmac/doc/interfaces.md > Hardware Interfaces

   - `hw/ip/hmac/doc/interfaces.md:5`: `Primary Clock: **`clk_i`**`

   Verified: No dedicated check that clk_i is the primary clock.

3. **Requirement** `cr_3627dd0601ce`: The hmac module has no peripheral pins for chip IO.
   Section: hw/ip/hmac/doc/interfaces.md > Hardware Interfaces

   - `hw/ip/hmac/doc/interfaces.md:9`: `Peripheral Pins for Chip IO: *none*`

   Verified: There are no pins, so nothing to test.

4. **Requirement** `cr_b923d18300df`: The hmac module has no other clocks.
   Section: hw/ip/hmac/doc/interfaces.md > Hardware Interfaces

   - `hw/ip/hmac/doc/interfaces.md:6`: `Other Clocks: *none*`

   Verified: Static statement; no test checks it.

5. **Requirement** `cr_2677d0527dcb`: hmac_done is an Event-type interrupt.
   Section: hw/ip/hmac/doc/interfaces.md > Hardware Interfaces > Interrupts

- `hw/ip/hmac/doc/interfaces.md:23`: `hmac_done`
- `hw/ip/hmac/doc/interfaces.md:23`: `Event`

Verified: The scoreboard checks hmac_done firing but not its Event type. This is weak and could arguably be withdrawn.

- `hw/ip/hmac/dv/env/hmac_scoreboard.sv:416`: `void'(ral.intr_state.hmac_done.predict(.value(1), .kind(UVM_PREDICT_READ)));` — Scoreboard predicts hmac_done interrupt state; it checks the interrupt firing, not its Event type.

Searched:

- hmac_done usage in DV in `hw/ip/hmac/dv`: smoke vseq, scoreboard and tb use it; no check for Event-type semantics
- TL-UL host interface checks (item 1) in `hw/ip/hmac/dv, hw/ip/hmac/doc/interfaces.md`: only the doc statement; no test checks it
- clk_i primary clock check (item 2) in `hw/ip/hmac/dv`: the env drives clk_i; no check that it is the primary clock
- chip IO pins (item 3) in `hw/ip/hmac/dv`: no pins exist, so nothing to test
- other clocks (item 4) in `hw/ip/hmac/dv`: no other clocks and no check

5.22 Only weak evidence (no check that can fail): 1 requirement in programmers_guide.md > Saving and restoring the context

`f_39ed7bdc4973` · severity medium

1. **Requirement** `cr_9b457cf2a7cd`: Context switches are possible only at the boundary of complete message blocks.

   - `hw/ip/hmac/doc/programmers_guide.md:144`: `Such context switches are possible only at the boundary of complete message blocks`

   Nearest evidence (not counted as a test): `save_and_restore` (stimulus only, no check)

   Verified: Only a positive stimulus at the boundary exists. No test tries a mid-block context switch, and no check would fail if one were allowed.

   - `hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:554-556`: `// Wait until message transmission is on a block boundary (multiple of 512 bits in SHA-2 256` — Stimulus only ever saves at a block boundary; no negative case.

Searched:

- save_and_restore, block boundary, mid-block, partial, unaligned in `hw/ip/hmac`: Only the boundary-wait in the vseq, a coverage group and the scoreboard's rounding of the message counter. No mid-block save test and no check for one.

Looked at and ruled out:

- `hw/ip/hmac/dv/env/hmac_scoreboard.sv:239`: Rounds the message counter up to the block boundary for prediction. This is not a check on the context-switch restriction.

5.23 No test found: 3 requirements in programmers_guide.md > Saving and restoring the context (and 2 more sections)

`f_854b37701406` · severity medium

1. **Requirement** `cr_fc9c477b91bc`: Key registers cannot be read from SW.
   Section: hw/ip/hmac/doc/programmers_guide.md > Programmer's Guide > Saving and restoring the context

   - `hw/ip/hmac/doc/programmers_guide.md:158`: `key registers cannot be read from SW`

   Verified: No test reads the key registers and checks the value.

2. **Requirement** `cr_4eab539dc535`: SwPushMsgWhenShaDisabled has ERR_CODE value 0x1.
   Section: hw/ip/hmac/doc/programmers_guide.md > Programmer's Guide > Errors

   - `hw/ip/hmac/doc/programmers_guide.md:179`: `SwPushMsgWhenShaDisabled`
   - `hw/ip/hmac/doc/programmers_guide.md:179`: `0x1`

   Verified: Code 1 is never predicted or checked, and its coverage bin is commented out.

   - `hw/ip/hmac/dv/env/hmac_env_cov.sv:109`: `// bins push_msg_when_sha_disabled = {SwPushMsgWhenShaDisabled};` — The code 0x1 bin is commented out, and the scoreboard never predicts code 1 (item 2).

3. **Requirement** `cr_9287ecbc9b5d`: fifo_empty status bit is lowered for a cycle even when the Message FIFO depth stays 0.
   Section: hw/ip/hmac/doc/programmers_guide.md > Programmer's Guide > FIFO Depth and Empty status

   - `hw/ip/hmac/doc/programmers_guide.md:191`: `The Message FIFO's `fifo_empty` status bit, however, is lowered for a cycle.`

   Verified: The one-cycle dip at depth 0 is not modelled or checked, and the smoke sequence says it can't be aligned.

   - `hw/ip/hmac/dv/env/seq_lib/hmac_smoke_vseq.sv:209`: `// there is one clk cycle difference between scb and design when predict fifo_empty,` — The fifo_empty one-cycle lowering is explicitly not aligned or checked (item 4).

Item 3 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- SwPushMsgWhenShaDisabled / code 1 use in `hw/ip/hmac/dv`: Only the enum definition and a commented-out coverage bin. No prediction or check.
- Key register read checks in `hw/ip/hmac/dv`: Only the key_length read. No test reads the key registers and checks the value.
- fifo_empty status dip at depth 0 in `hw/ip/hmac/dv/env`: The scoreboard predicts hmac_fifo_empty = (depth == 0), with no lowered-for-a-cycle behaviour. The smoke vseq comments that the cycle can't be aligned.
- push msg when disallowed in `hw/ip/hmac/dv`: Scoreboard line 148 plus hmac_error_vseq. Tested.

Looked at and ruled out:

- `hw/ip/hmac/dv/env/hmac_scoreboard.sv:147-148`: Folds the !sha_en case into code 5 rather than code 1, so it does not verify item 2.

5.24 No test found: 5 requirements in programmers_guide.md > Triggering HMAC/SHA-2 engine (and 1 more section)

`f_e9ce8dbfdcba` · severity medium

1. **Requirement** `cr_09387e293565`: For a message larger than 512 bits, software waits until the FIFO is not full before writing further bits.
   Section: hw/ip/hmac/doc/programmers_guide.md > Programmer's Guide > Triggering HMAC/SHA-2 engine

   - `hw/ip/hmac/doc/programmers_guide.md:42`: `If the message is larger than 512-bit, the software must wait until the FIFO is not full before writing further bits.`

   Verified: This is a software rule. The DV checks the FIFO-full status bit but nothing enforces or checks the wait-before-write behavior.

2. **Requirement** `cr_6fdc529bb101`: CFG.sha_en is set only after all other configuration inputs have been set.
   Section: hw/ip/hmac/doc/programmers_guide.md > Programmer's Guide > Triggering HMAC/SHA-2 engine

   - `hw/ip/hmac/doc/programmers_guide.md:41`: `once all other configuration inputs have been properly set`

   Verified: hmac_init writes all of CFG in a single write, and nothing checks the ordering of sha_en against the other configuration.

3. **Requirement** `cr_03c30ada2ff4`: If the key manager revokes the key during an operation, the operation is aborted.
   Section: hw/ip/hmac/doc/programmers_guide.md > Programmer's Guide > Using a sideloaded key

   - `hw/ip/hmac/doc/programmers_guide.md:111`: `If the key manager revokes the key while an operation is running, that operation is aborted`

   Verified: Sideload is disabled in the DV and no test revokes the key during an operation. Only RTL assertions exist.

   - `hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:211-212` — Random configs force sideload off, so no sideload test runs. This covers items 3, 4 and 5.

     ```
     // TODO(#31634): Disable sideloading for now.
       rand_config_value[ral.cfg.sideload.get_lsb_pos()] = 1'b0;
     ```

   - `hw/ip/hmac/rtl/hmac.sv:1061-1062` — RTL assertions exist, but nothing in the DV makes the key invalid mid-operation. They do not check that the message and intermediate hash are discarded, or that a key change is handled.

```
`ASSERT(SideloadKeyInvalidBlocksStart_A, sideload_key_invalid |->
!hash_start_or_continue)
  `ASSERT(SideloadKeyInvalidAborts_A, sideload_key_invalid |-> !sha_en_engine)
```

4. **Requirement** `cr_2afcaa91c549`: After an abort, the part of the message already written is discarded together with the intermediate hash.
   Section: hw/ip/hmac/doc/programmers_guide.md > Programmer's Guide > Using a sideloaded key

   - `hw/ip/hmac/doc/programmers_guide.md:125`: `The part of the message that had been written is discarded together with the intermediate hash, and the context cannot be restored with a sideloaded key.`

   Verified: No abort and restart scenario is tested.

5. **Requirement** `cr_371670d7ec74`: The sideloaded key must remain unchanged for the entire operation.
   Section: hw/ip/hmac/doc/programmers_guide.md > Programmer's Guide > Using a sideloaded key

   - `hw/ip/hmac/doc/programmers_guide.md:107`: `The key also has to remain unchanged for the entire operation.`

   Verified: No test changes the sideloaded key during an operation.

Searched:

- item cr_09387e293565: FIFO-full waiting (fifo_full, wait_fifo) in the DV in `hw/ip/hmac/dv/env`: The scoreboard tracks the FIFO-full status bit and the coverage samples it. Nothing checks that software waits before writing.
- item cr_6fdc529bb101: sha_en ordering (sha_en) in the DV sequences in `hw/ip/hmac/dv/env/seq_lib`: hmac_init writes CFG in one go. Other sequences toggle sha_en for stop/continue and invalid-config cases, not for configuration ordering.
- item cr_03c30ada2ff4: key_valid, revoke and key_invalid in the DV in `hw/ip/hmac`: Only RTL matches (hmac.sv abort logic and assertions). No DV stimulus drives the key invalid, and the sideload agent has start_default_seq=0.
- item cr_2afcaa91c549: abort and restart, message discarded in `hw/ip/hmac/dv`: Nothing found. The scoreboard's SwInvalidConfig handling relates only to invalid CFG.
- item cr_371670d7ec74: key unchanged during an operation, sideload sequences in `hw/ip/hmac/dv/env/seq_lib`: Sideload is forced to 0 in hmac_base_vseq.sv. No test changes the key mid-operation.

Looked at and ruled out:

- `hw/ip/hmac/dv/env/hmac_env_cov.sv:96-101`: Covers the FIFO-full status bit only. It does not check that software waits for the FIFO to drain before writing.

5.25 No test found: 5 requirements in programmers_guide.md > Using a sideloaded key

`f_743126387071` · severity medium

1. **Requirement** `cr_4311569af12c`: If the key is revoked during an operation, the digest is cleared.

   - `hw/ip/hmac/doc/programmers_guide.md:111`: `the digest is cleared`

Verified: No test revokes the key during an operation.

- `hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:211-212`: `// TODO(#31634): Disable sideloading for now.` — Sideload is disabled in randomized config.
- `hw/ip/hmac/dv/env/hmac_scoreboard.sv:307`: `exp_digest = '{default:0}; // Digest gets cleared on disabling.` — Digest clearing is checked only when SHA is disabled, not on key revocation.

2. **Requirement** `cr_90db0cc9a415`: The CFG configuration is retained across an abort.

   - `hw/ip/hmac/doc/programmers_guide.md:127`: `The configuration in CFG is retained across an abort and does not need to be written again.`

   Verified: No abort-related test or check found.

3. **Requirement** `cr_bd802d2e0596`: The valid indication of the sideload interface must remain set for the entire operation.

   - `hw/ip/hmac/doc/programmers_guide.md:106`: `This indication has to remain set for the entire operation.`

   Verified: Sideload is disabled, so key valid is never exercised.

4. **Requirement** `cr_c4e4f4d945ab`: CFG.sideload is ignored when only SHA-2 is computed (no key is used); it applies only to keyed HMAC operations.

   - `hw/ip/hmac/doc/theory_of_operation.md:135`: `it is ignored when only SHA-2 is computed, as that does not use a key.`
   - `hw/ip/hmac/doc/programmers_guide.md:78`: `This only applies to keyed HMAC operations; it is ignored when only SHA-2 is computed.`

   Verified: CFG.sideload is always 0.

5. **Requirement** `cr_c8ea510e0e76`: A key becoming invalid after operation completion clears the digest.

   - `hw/ip/hmac/doc/programmers_guide.md:116`: `A key that becomes invalid after the operation has completed clears the digest`
   - `hw/ip/hmac/doc/programmers_guide.md:116`: `so the digest registers then silently read as zero`

   Verified: No test invalidates the key after completion.

Searched:

- cr_4311569af12c: revoke/key_valid/sideload stimulus during an operation in `hw/ip/hmac/dv`: Only agent wiring and sideload disabled; no revocation stimulus.
- cr_90db0cc9a415: abort / CFG retained in `hw/ip/hmac/dv (case-insensitive 'abort')`: No abort-related test or check.
- cr_bd802d2e0596: key valid held for the whole operation in `hw/ip/hmac/dv`: key_sideload agent is never started (start_default_seq = 0) and sideload is disabled, so no check.
- cr_c4e4f4d945ab: CFG.sideload ignored for SHA-2-only in `hw/ip/hmac/dv`: sideload bit is always forced to 0, so the case is never exercised.

- cr_c8ea510e0e76: digest cleared when key becomes invalid after completion in `hw/ip/hmac/dv (digest clear/zero, key_invalid)`: Only the SHA-disable digest clear in the scoreboard and an unrelated coverage bin.

5.26 No test found: 4 requirements in programmers_guide.md > Using a sideloaded key (and 1 more section)

`f_a3679f6039c0` · severity medium

1. **Requirement** `cr_cf99e411151d`: No error is reported when the key becomes invalid after the operation has completed.
   Section: hw/ip/hmac/doc/programmers_guide.md > Programmer's Guide > Using a sideloaded key

   - `hw/ip/hmac/doc/programmers_guide.md:116`: `and no error is reported in this case`

   Verified: Sideload is disabled in DV, so no test makes the key invalid after completion.

   - `hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:211-212` — Sideload is never enabled in DV, so the key-invalid behaviours (items 1 and 2) are never stimulated.

     ```
     // TODO(#31634): Disable sideloading for now.
       rand_config_value[ral.cfg.sideload.get_lsb_pos()] = 1'b0;
     ```

2. **Requirement** `cr_ea4b652641c0`: While INTR_STATE.hmac_err is pending, no further error is reported in ERR_CODE.
   Section: hw/ip/hmac/doc/programmers_guide.md > Programmer's Guide > Using a sideloaded key

   - `hw/ip/hmac/doc/programmers_guide.md:121`: `As long as it is pending, no further error is reported in ERR_CODE.`

   Verified: The scoreboard has the no-overwrite logic, but sideload key errors are not stimulated, so the sideloaded-key case is untested.

   - `hw/ip/hmac/dv/env/hmac_scoreboard.sv:1041-1043` — The model only updates ERR_CODE when hmac_err is not already pending. This is in the scoreboard, but sideload key errors are never driven, so item 2 stays untested for the sideloaded-key case.

     ```
     task hmac_scoreboard::update_err_intr_code(err_code_e err_code_val);
       if (!`gmv(ral.intr_state.hmac_err)) begin
     ```

3. **Requirement** `cr_51fc8f2304d3`: Enabling sha_en locks down SW writes to the DIGEST_0..15 registers.
   Section: hw/ip/hmac/doc/programmers_guide.md > Programmer's Guide > Saving and restoring the context

   - `hw/ip/hmac/doc/programmers_guide.md:166`: `locks down writing to the DIGEST_0..15 registers from SW.`

   Verified: The model supports the lock, but no directed stimulus writes DIGEST while sha_en is set and checks the write was ignored.

   - `hw/ip/hmac/dv/env/hmac_scoreboard.sv:338-341` — The model encodes the write lock, but no directed stimulus writes DIGEST while sha_en is set and then checks that the write was ignored.

```
// Predict updated value coming from write iff SHA core is disabled.
          do_predict = !sha_en;
```

4. **Requirement** `cr_d2c659d4c60b`: Each message stream is finalized once by setting CMD.hash_process.
   Section: hw/ip/hmac/doc/programmers_guide.md > Programmer's Guide > Saving and restoring the context

   - `hw/ip/hmac/doc/programmers_guide.md:149`: `finalized *once* by setting the CMD.hash_process bit`

   Verified: No test or check covers finalizing each stream exactly once.

Item 3 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- sideload / key-invalid / err_code in hmac dv in `hw/ip/hmac/dv`: Sideload is forced to 0 in the base vseq. No key-invalid error test exists.
- digest clear and digest write lock in `hmac_scoreboard.sv and seq_lib`: The clear on sha_en 1->0 is modelled and compared. csr_wr_digest is only used in the save-and-restore flow, with no write-while-enabled check.
- hash_process finalized once in `seq_lib`: No test checks that a stream is finalized only once.

5.27 No test found: 5 requirements in theory_of_operation.md > Block Diagram (and 2 more sections)

`f_1150de25abf8` · severity medium

1. **Requirement** `cr_b09d206cb6b3`: The SHA-2 message scheduling FIFO is not software accessible.
   Section: hw/ip/hmac/doc/theory_of_operation.md > Theory of Operation > Block Diagram

   - `hw/ip/hmac/doc/theory_of_operation.md:17`: `The message scheduling FIFO is not software accessible`

   Verified: No test attempts or checks software access to the scheduling FIFO.

2. **Requirement** `cr_fd137b45d0d9`: The SHA-2 hash engine is a SHA-2 engine primitive instantiated with multi-mode feature enabled.
   Section: hw/ip/hmac/doc/theory_of_operation.md > Theory of Operation > Block Diagram

   - `hw/ip/hmac/doc/theory_of_operation.md:7`: `which is a SHA-2 engine primitive instantiated with the multi-mode feature enabled`

   Verified: Multi-mode instantiation appears only in coverage exclusions, with no check.

   - `hw/ip/hmac/dv/cov/hmac_unr_excl.el:13`: `INSTANCE: tb.dut.u_prim_sha2_512.gen_multimode_logic.u_prim_sha2_multimode.u_pad` — Multi-mode primitive appears only in coverage exclusions, not in checks.

3. **Requirement** `cr_6241cc5595f4`: Packed words are fed into the internal message scheduling FIFO.
   Section: hw/ip/hmac/doc/theory_of_operation.md > Theory of Operation > Design Details > SHA-2 message feed and pad

- `hw/ip/hmac/doc/theory_of_operation.md:57`: `These words are fed into the internal message scheduling FIFO.`

Verified: No check on the packer-to-scheduling-FIFO path beyond end-to-end digest comparison.

- `hw/ip/hmac/doc/theory_of_operation.md:56`: `Small writes to [`MSG_FIFO`](registers.md#msg_fifo) are coalesced into 32-bit words` — Packer coalescing is only exercised indirectly by end-to-end digest comparison.

4. **Requirement** `cr_e937e36d780f`: Small writes to MSG_FIFO are coalesced into 32-bit words by the packer logic.
   Section: hw/ip/hmac/doc/theory_of_operation.md > Theory of Operation > Design Details > SHA-2 message feed and pad

   - `hw/ip/hmac/doc/theory_of_operation.md:56`: `Small writes to MSG_FIFO are coalesced into 32-bit words by the packer logic.`

   Verified: Coalescing into 32-bit words is not checked directly. Only digests are compared.

5. **Requirement** `cr_236bbacf0c98`: In each round the compression function fetches a 64-bit word from the buffer and computes the internal variables.
   Section: hw/ip/hmac/doc/theory_of_operation.md > Theory of Operation > Design Details > SHA-2 computation

   - `hw/ip/hmac/doc/theory_of_operation.md:83`: `In each round, the compression function fetches a 64-bit word from the buffer and computes the internal variables.`

   Verified: No check on the per-round 64-bit word fetch in the compression function.

Searched:

- software access to the scheduling FIFO in `hw/ip/hmac/dv`: No test or check for it.
- multimode instantiation checks in `hw/ip/hmac/dv`: Only coverage exclusion entries, no assertion or check.
- packer to scheduling FIFO feeding in `hw/ip/hmac/dv`: Only the scoreboard's MSG_FIFO modelling. No check on the internal path.
- packer coalescing into 32-bit words in `hw/ip/hmac/dv`: Only end-to-end digest comparison.
- per-round 64-bit word fetch in the compression function in `hw/ip/hmac/dv`: No reference.

Looked at and ruled out:

- `hw/ip/hmac/dv/env/seq_lib/hmac_smoke_vseq.sv:211`: A comment about the sha2_pad state transition. It does not check packer coalescing or the scheduling FIFO.

5.28 No test found: 5 requirements in theory_of_operation.md > Key sideloading

`f_a9133b1841ac` · severity medium

1. **Requirement** `cr_291fea02ab56`: The key manager replaces the sideloaded key when a new key is generated for HMAC without deasserting the valid indication.

- `hw/ip/hmac/doc/theory_of_operation.md:148`: `the key manager replaces the sideloaded key whenever a new key is generated for HMAC, without deasserting the valid indication`

Verified: Neither DV nor the firmware test replaces the key mid-operation.

- `hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:211-212`: `// TODO(#31634): Disable sideloading for now.` — The DV forces sideload off in randomized configs.
- `sw/device/tests/keymgr_dpe_sideload_hmac_test.c:214-216`: `// Starting HMAC with an invalid sideload key must be blocked.` — The firmware test checks the invalid-key start case, which is not one of the five items.

2. **Requirement** `cr_2bd90963e6d1`: On abort, the SHA-2 engine is disabled, clearing its internal state and the digest.

   - `hw/ip/hmac/doc/theory_of_operation.md:156`: `The SHA-2 engine is disabled, which clears its internal state and the digest`

   Verified: No test checks that the digest or SHA-2 state is cleared after an abort. The firmware test only covers a blocked start.

3. **Requirement** `cr_4fa240172006`: The key manager keeps the valid indication set for the entire duration of an operation.

   - `hw/ip/hmac/doc/theory_of_operation.md:146`: `The key manager has to keep it set for the entire duration of an operation`
   - `hw/ip/hmac/doc/theory_of_operation.md:146`: `because the HMAC core uses the key twice: for the inner key padding when the operation starts, and again for the outer key padding after the message has been hashed.`

   Verified: No test checks the valid indication across an operation. DV has sideload disabled.

4. **Requirement** `cr_562194d14c29`: CFG.key_swap has no effect on the sideloaded key.

   - `hw/ip/hmac/doc/theory_of_operation.md:143`: `CFG.key_swap has no effect on the sideloaded key, as the key manager delivers it in a fixed order.`

   Verified: The scoreboard applies key_swap only to register keys, and sideload is off in DV.

5. **Requirement** `cr_7eee2f253918`: The combined sideloaded key is muxed into the HMAC core and no copy of it is kept inside HMAC.

   - `hw/ip/hmac/doc/theory_of_operation.md:139`: `The combined key is muxed into the HMAC core rather than latched, so no copy of the sideloaded key is kept inside HMAC`

   Verified: No test checks that HMAC keeps no copy of the key.

Searched:

- sideload/key_valid/key_swap in the hmac DV env, scoreboard and sequences in `hw/ip/hmac/dv`: Sideload is disabled in the base vseq. The scoreboard applies key_swap only to register keys. No sideload checks.
- sideload references in hmac hjson, sv, md and core files in `hw/ip/hmac`: There is a key_sideload agent and a tb interface, but sideload is forced off. The sec_cm testplan has a sec_cm_key_sideload entry.

- HMAC sideload firmware tests in `sw/device/tests`: keymgr_dpe_sideload_hmac_test.c checks the SwInvalidConfig error for an invalid key. It does not cover key replacement, abort clearing of SHA-2 state, valid held for a whole operation, key_swap on a sideloaded key, or the absence of a key copy.

Looked at and ruled out:

- `hw/ip/hmac/dv/env/hmac_scoreboard.sv:861-862`: key_swap is modelled for register keys only.

5.29 No test found: 5 requirements in theory_of_operation.md > Key sideloading

`f_bcc8e0e600a3` · severity medium

1. **Requirement** `cr_89cd9ce85cff`: CFG.sideload only takes effect for keyed HMAC operations, that is when CFG.hmac_en is set.

   - `hw/ip/hmac/doc/theory_of_operation.md:135`: `The bit only takes effect for keyed HMAC operations (CFG.hmac_en set)`

   Verified: Sideload is forced to 0, so no stimulus reaches it.

   - `hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:211-212` — Sideload is forced off in randomized config.

     ```
     // TODO(#31634): Disable sideloading for now.
     rand_config_value[ral.cfg.sideload.get_lsb_pos()] = 1'b0;
     ```

2. **Requirement** `cr_aeeb0cb3e0f0`: On abort, the message packer and the message FIFO are cleared.

   - `hw/ip/hmac/doc/theory_of_operation.md:156`: `the message packer and the message FIFO are cleared so that no residual message data is carried into the next operation`

   Verified: No abort stimulus exists, since sideload is disabled.

3. **Requirement** `cr_af28a6129f5c`: The key manager deasserting the valid indication during an operation aborts that operation.

   - `hw/ip/hmac/doc/theory_of_operation.md:155`: `The key manager deasserting the valid indication while an operation is in progress aborts that operation`

   Verified: No test deasserts valid during an operation.

   - `hw/ip/hmac/dv/env/hmac_env_cfg.sv:58`: `keymgr_sideload_agent_cfg.start_default_seq = 0;` — Sideload agent default sequence is not started.

4. **Requirement** `cr_b276172d406a`: If the sideloaded key becomes invalid (key revoked / valid deasserted) while an operation is in progress, the operation is aborted and ERR_CODE reports SwInvalidConfig.

   - `hw/ip/hmac/doc/programmers_guide.md:111`: `and the same error is reported`
   - `hw/ip/hmac/doc/programmers_guide.md:186`: `It is also reported for the sideloaded key`
   - `hw/ip/hmac/doc/programmers_guide.md:186`: `when the key becomes invalid while an operation is in progress`
   - `hw/ip/hmac/doc/theory_of_operation.md:155`: `and reports the same error.`

Verified: No test drives key invalidation, so ERR_CODE is never checked for this case.

5. **Requirement** `cr_d882830deff5`: HMAC does not detect a change of the sideloaded key during an operation.

   - `hw/ip/hmac/doc/theory_of_operation.md:148`: `Neither HMAC nor the key manager enforces this`
   - `hw/ip/hmac/doc/theory_of_operation.md:148`: `and HMAC does not detect the change.`

   Verified: No test changes the sideloaded key mid-operation.

Searched:

- sideload / CFG.sideload with hmac_en (item cr_89cd9ce85cff) in `hw/ip/hmac/dv`: Only forced to 0 in base vseq; no test sets it.
- abort clears packer and FIFO (cr_aeeb0cb3e0f0): sideload, key_valid, abort in `hw/ip/hmac/dv`: No abort stimulus or checker.
- key valid deassert mid-operation (cr_af28a6129f5c): key_valid, sideload in `hw/ip/hmac/dv`: Nothing drives valid; agent default sequence disabled.
- SwInvalidConfig on key invalid (cr_b276172d406a) in `hw/ip/hmac/dv`: Scoreboard SwInvalidConfig handling at lines 199 and 219 covers other cases; sideload never enabled.
- key change during operation (cr_d882830deff5): sideload key change in `hw/ip/hmac/dv`: No sequence changes the sideload key.

Looked at and ruled out:

- `hw/ip/hmac/dv/env/hmac_scoreboard.sv:199-219`: SwInvalidConfig checks exist, but they cover other configuration errors, and sideload is never enabled to trigger them.

5.30 No test found: 4 requirements in theory_of_operation.md > Performance in SHA-2 mode and HMAC mode

`f_a24982bebf63` · severity medium

1. **Requirement** `cr_2a29e638293c`: The next 16 x 32-bit message is fetched into the internal message scheduling array only after the current block completes.

   - `hw/ip/hmac/doc/theory_of_operation.md:179`: `This version of the IP fetches the next 16 x 32-bit message into the internal message scheduling array only after completing the current block.`

   Verified: No test checks the fetch-after-block-completion timing. It is only implied by the 65-cycle model.

   - `hw/ip/hmac/dv/env/hmac_env_pkg.sv:37-39`: `// 48 cycles of hashing, 16 cycles to rd next 16 words, 1 cycle to update digest` — The DV model uses 65 and 81 cycles per block, not the 80 and 96 the spec states. No exact-latency check exists.

2. **Requirement** `cr_aeb18e72aa7a`: SHA-2 256 takes 80 cycles to complete a block.

   - `hw/ip/hmac/doc/theory_of_operation.md:180`: `As such, it takes 80 cycles to complete a block.`

Verified: The DV model uses 65 cycles per block and never checks 80. No direct latency check exists.

3. **Requirement** `cr_ba453872049c`: SHA-2 384/512 takes 96 cycles to complete a 1024-bit block.

   - `hw/ip/hmac/doc/theory_of_operation.md:185`: `It takes 96 cycles to complete a 1024-bit block.`

   Verified: The DV model uses 81 cycles for SHA-2 384/512, not 96. No check of 96 exists.

4. **Requirement** `cr_ed89d1bd86c3`: HMAC mode adds an extra 240 cycles to complete a HMAC SHA-2 256 digest.

   - `hw/ip/hmac/doc/theory_of_operation.md:190`: `This adds an extra 240 cycles (80 for the inner key, 80 for the outer key, and 80 for the result of the first round) to complete a HMAC SHA-2 256 digest of a message.`

   Verified: Only the key-padding wait is modelled. The 240-cycle total HMAC overhead is never measured or checked.

Item 4 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- cycle-count checks (80, 96, 240, block cycles) in hmac dv and non-doc files in `hw/ip/hmac`: Only the env_pkg parameters (65, 81, 80 and 112 for key padding) and the scoreboard wait models. No assertion or check of the 80, 96 or 240 cycle figures.
- FIFO full / empty modelling in `hw/ip/hmac/dv/env/hmac_scoreboard.sv`: The model predicts status.fifo_full and the fifo_empty interrupt with a block-drain delay. This indirectly checks item 4.

Looked at and ruled out:

- `hw/ip/hmac/dv/env/hmac_scoreboard.sv:770-780`: Models 80-cycle key padding for HMAC, but this is neither the 240-cycle total overhead nor a check of the DUT's latency.

5.31 No test found: 1 requirement in theory_of_operation.md > Performance in SHA-2 mode and HMAC mode (and 1 more section)

`f_ff609afdb8eb` · severity medium

1. **Requirement** `cr_f6528a1e1fc7`: SHA-2 256 effective throughput is 64 bytes per 80 clocks, 20% of the maximum throughput.
   Section: hw/ip/hmac/doc/theory_of_operation.md > Theory of Operation > Design Details > Performance in SHA-2 mode and HMAC mode

   - `hw/ip/hmac/doc/theory_of_operation.md:181`: `The effective throughput considering this is \`64 byte / 80 clk\` or \`16 clk / 80 clk\`, 20% of the maximum throughput.`

   Verified: No test checks the 64 bytes per 80 clocks timing.

   - `hw/ip/hmac/dv/env/hmac_scoreboard.sv:414`: `// future throughput test should check the accurate cycles` — Throughput cycles are not checked.

Item 2 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- throughput / 80 clk / cycle-count checks in `hw/ip/hmac`: Only the scoreboard comment saying a future throughput test is needed.
- MSG_FIFO window address stimulus and checks in `hw/ip/hmac/dv`: wr_addr is constrained to the whole window, and the scoreboard handles window writes.

5.32 No test found: 2 requirements in theory_of_operation.md > SHA-2 computation (and 2 more sections)

`f_21c95b4eed04` · severity medium

4. **Requirement** `cr_003bb8525faa`: On abort, the HMAC core returns to its idle state.
   Section: hw/ip/hmac/doc/theory_of_operation.md > Theory of Operation > Design Details > Key sideloading

   - `hw/ip/hmac/doc/theory_of_operation.md:156`: `and the HMAC core returns to its idle state.`

   Verified: Sideload is disabled in DV and no abort test exists.

   - `hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:211-212`: `// TODO(#31634): Disable sideloading for now.` — Sideload is disabled in randomized config, so sideload abort and key stability are never stimulated.

5. **Requirement** `cr_01801357ac87`: The sideloaded key stays stable for the entire duration of an operation.
   Section: hw/ip/hmac/doc/theory_of_operation.md > Theory of Operation > Design Details > Key sideloading

   - `hw/ip/hmac/doc/theory_of_operation.md:147`: `For the same reason, the key itself has to stay stable for the entire duration of an operation.`

   Verified: No sideload stimulus or key stability check exists, since sideload is disabled.

Items 1, 2, 3 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- sideload/abort/SwInvalidConfig in the DV env, tb and sva in `hw/ip/hmac/dv`: Sideload is forced to 0 in the base vseq. The sideload agent is created but has no sequence. The sva directory has only TLUL and CSR asserts.
- key_swap checks in `hw/ip/hmac/dv/env/hmac_scoreboard.sv`: The expected HMAC is computed with and without key swap.
- FIFO fetch during compression and depth limit in `hw/ip/hmac/dv`: Only a status FIFO depth/full model and coverage. No explicit check of the cited statements.

Looked at and ruled out:

- `hw/ip/hmac/dv/env/hmac_scoreboard.sv:199-219`: SwInvalidConfig is modeled for other config errors, not for a sideload key becoming invalid.

Inconclusive

The verifier could neither confirm nor refute these. They need a human decision.

- `f_3fc7728836ce` (no test found) CFG.digest_swap: the register is implemented outside the register block (true). — digest_swap is driven by the smoke tests and checked by the scoreboard. hwext is a structural attribute and is only covered indirectly.
- `f_3fc7728836ce` (no test found) CFG.sha_en: hardware receives a write-qualifier pulse on software writes (true). — sha_en writes are exercised and the scoreboard tracks them. The hwqe attribute is covered only indirectly.
- `f_3fc7728836ce` (no test found) CFG.sha_en: the register is implemented outside the register block (true). — sha_en is exercised through CFG writes. The hwext attribute is covered only indirectly.
- `f_3fc7728836ce` (no test found) CFG.hmac_en: hardware receives a write-qualifier pulse on software writes (true). — hmac_en writes are exercised by the smoke tests and the scoreboard checks the results. The hwqe attribute is covered only indirectly.
- `f_81c7d089aa21` (partly tested) With SHA2_None, HMAC/SHA-2 is blocked from starting when triggered via hash_start or hash_continue. — The scoreboard does check idle state for an invalid config on the process path. I could not confirm that a hash_continue trigger is exercised.
- `f_dcd02c1afd9c` (no test found) The message length has byte granularity. — The scoreboard predicts the length as a byte count times 8, and the SAR check compares the bit length against msg size in bytes. Byte granularity is only implicitly checked.
- `f_21c95b4eed04` (no test found) Software can push up to 16 entries (32 for SHA-2 384/512) to the FIFO for the next hash computation. — The scoreboard models FIFO depth and full against the 16/32 entry limit, but nothing explicitly checks pushing during compression.
- `f_21c95b4eed04` (no test found) During compression rounds the engine does not fetch data from the message FIFO. — No explicit check that the engine does not fetch from the FIFO during compression. The FIFO depth model gives only indirect coverage.
- `f_6e3c88ec1c93` (partly tested) HMAC.KEY_LENGTH.KEY_512 supports HMAC with any digest size using a 512-bit key. — Randomized only; cross coverage exists but closure is not shown.
- `f_6e3c88ec1c93` (partly tested) HMAC.KEY_LENGTH.KEY_1024 supports HMAC based on SHA-2 384 and SHA-2 512 using a 1024-bit key. — Random plus vectors; no directed guarantee.
- `f_6e3c88ec1c93` (partly tested) HMAC.KEY_LENGTH.KEY_256 supports HMAC with any digest size using a 256-bit key. — Random only.
- `f_6e3c88ec1c93` (partly tested) With Key_None, HMAC is blocked from starting when triggered via hash_start or hash_continue. — hash_start with Key_None is covered by random stimulus. The hash_continue trigger is not confirmed.
- `f_6050313b3ebc` (test plan items with no test) regwen_val_when_new_value_written_cg — It is a covergroup, not a test; no test is expected for it
- `f_6050313b3ebc` (test plan items with no test) NO_ISSUES_PENDING — It is a checklist status entry, not a testable item
- `f_b04480aeaeba` (test plan items with no test) NO_TOOL_WARNINGS_THROWN — This is a process checklist item, not a test, so there is no test to find.
- `f_b04480aeaeba` (test plan items with no test) FPV_COI_COVERAGE_AT_100 — The item is marked N/A for HMAC, so no test is expected. This is not a test gap.

Withdrawn

Candidates the verifier refuted, with the reason. Listed so the refutations can be checked too.

- `f_b03fa03996ed` (spec too ambiguous to test) In HMAC mode, start is blocked when key_length is Key_1024 and digest_size is SHA2_256. — The programmer's guide answers the main question. (`hw/ip/hmac/doc/programmers_guide.md:174-175`, `hw/ip/hmac/doc/programmers_guide.md:186`)
- `f_248d8da64e01` (spec too ambiguous to test) CPU must configure digest size, secret key and key length before starting. — Other HMAC spec text answers the question. (`hw/ip/hmac/doc/programmers_guide.md:186`, `hw/ip/hmac/doc/registers.md:212`)
- `f_13ebd67552c8` (spec too ambiguous to test) Software may write the DIGEST registers when STATUS.hmac_idle is 1. — The programmer's guide gives the missing condition. (`hw/ip/hmac/doc/programmers_guide.md:165-166`, `hw/ip/hmac/doc/programmers_guide.md:163-164`)
- `f_026e7be916d9` (spec too ambiguous to test) HMAC.SECURE_WIPE wipes the internal state and secret key with a constant value provided by software. — The README and the WIPE_SECRET register description say what is wiped and with what value. (`hw/ip/hmac/README.md:59-63`, `hw/ip/hmac/data/hmac.hjson:455-457`)
- `f_59efb75c18ac` (spec too ambiguous to test) SwInvalidConfig is reported for an invalid digest size for SHA-2/HMAC modes. — The register description in hmac.hjson answers the question. (`hw/ip/hmac/data/hmac.hjson:269-270`, `hw/ip/hmac/data/hmac.hjson:294-295`)
- `f_69f49c5e5f17` (spec too ambiguous to test) SwInvalidConfig is reported for an invalid key length in HMAC mode. — The register description for CFG.key_length answers the question: in HMAC mode, Key_None (including unsupported/invalid and all-zero encodings) and Key_1024 with SHA2_256 block the start and signal an error. (`hw/ip/hmac/doc/registers.md:203`, `hw/ip/hmac/doc/registers.md:212`)
- `f_e96e54c35582` (spec too ambiguous to test) For SHA-2 384, only DIGEST_0..11 should be read out, the rest should be truncated out by not being read by the SW. — The DIGEST register description says that for SHA-2 384, DIGEST12..15 are truncated, irrelevant and should not be read out. (`hw/ip/hmac/doc/registers.md:423`, `hw/ip/hmac/doc/theory_of_operation.md:25`)
- `f_0c2b65414c0f` (spec too ambiguous to test) prim_packer compacts written data into word size and writes it to the MSG_FIFO when the write is not a full word write. — Other spec text answers most of the question. (`hw/ip/prim/doc/prim_packer.md:24`, `hw/ip/prim/doc/prim_packer.md:103`)
- `f_345252d6d22e` (spec contradicts itself) The row states the error is reported on a MSG_FIFO write while SHA is disabled, yet also says the code is not used in th — The row describes the nominal condition for 0x1 and then says the code is not used in the current IP, and that SwPushMsgWhenDisallowed (0x5) is reported instead. (`hw/ip/hmac/doc/programmers_guide.md:179`)
- `f_1dac27e3262f` (test contradicts its own description) hmac_smoketest: plan vs. test — The plan item (chip_sw_hmac_enc) is linked to hmac_enc_test, not to hmac_smoketest. (`hw/top_earlgrey/data/ip/chip_hmac_testplan.hjson:19-21`)
- `f_f6592776203d` (test contradicts its own description) hmac_error_conditions_test: comment vs. code — The programmer's guide says SwPushMsgWhenShaDisabled (0x1) is not used in the current IP. (`hw/ip/hmac/doc/programmers_guide.md:179`)
- `f_349d3e01b978` (test contradicts its own description) keymgr_dpe_sideload_hmac_test: plan vs. test — The repository does not link the HMAC plan item to this test. (`hw/ip/hmac/data/hmac_sec_cm_testplan.hjson:34-36`)
- `f_3747f22ce1b8` (test contradicts its own description) check_error_code: comment vs. code — The write-back exists at line 543: csr_wr writes error_code, which was just read from intr_state, back to clear it. (`hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:541-544`)

- `f_93f9ac580c2e` (test contradicts its own description) hmac_scoreboard::run_phase: comment vs. code — The code does wait for reset assertion (line 88) and then for release (line 90, at the top of the forever loop). (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:88-90`)
- `f_8f147aa0fc29` (spec not tested) Partially tested: 5 requirement(s) in hw/ip/hmac/data/hmac.hjson > hmac > CFG — The hmac DV environment covers all five items. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:359-373`, `hw/ip/hmac/dv/env/hmac_scoreboard.sv:157-163`)
- `f_fb595bd4fab8` (spec not tested) 5 plan item(s) with no implementing test — All five items are false positives. (`hw/ip/hmac/dv/sva/hmac_bind.sv:7-14`, `hw/dv/tools/dvsim/tests/tl_access_tests.hjson:20-23`)
- `f_db9162be6462` (spec not tested) Only weak evidence (non-failing checks, coverage, list entries): 1 requirement(s) in hw/ip/hmac/README.md > Overview > Description — The scoreboard predicts SwInvalidConfig for hmac_en with SHA2_256 and Key_1024, and the CSR checks compare it against the DUT's err_code and intr_state. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:173-177`, `hw/ip/hmac/dv/env/hmac_scoreboard.sv:198-199`)
- `f_45c6516939b2` (spec not tested) Planned but not implemented: 3 requirement(s) in hw/ip/hmac/README.md > Overview > Features … (2 sections) — The HMAC DV environment has stimulus and failing checks for all three items. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:173-177`, `hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:449-451`)
- `f_a186a50affdd` (spec not tested) Untested: 5 requirement(s) in hw/ip/hmac/data/hmac.hjson > hmac > CFG — The items are register-implementation attributes (hwext, hwqe) on CFG. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:160`, `hw/ip/hmac/dv/env/hmac_scoreboard.sv:173`)
- `f_d5a5a5a9aee3` (spec not tested) Untested: 5 requirement(s) in hw/ip/hmac/data/hmac.hjson > hmac > CMD — The HMAC DV env drives CMD.hash_start, hash_process, hash_stop and hash_continue in its vseqs, and the scoreboard predicts and checks digest, done interrupt, idle status and error codes from those writes. (`hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:233-235`, `hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:248-250`)
- `f_32dca00488d7` (spec not tested) Untested: 5 requirement(s) in hw/ip/hmac/data/hmac.hjson > hmac > CMD … (2 sections) — hwext and hwqe are implementation attributes of the register interface, not separately testable behaviours. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:383-387`, `hw/ip/hmac/dv/env/hmac_scoreboard.sv:171-193`)
- `f_76363c75b0ec` (spec not tested) Untested: 2 requirement(s) in hw/ip/hmac/data/hmac.hjson > hmac > MSG_LENGTH_UPPER — Both items are exercised by the HMAC DV environment. (`hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:700-707`, `hw/ip/hmac/dv/env/hmac_scoreboard.sv:1035-1038`)
- `f_e52678ebec48` (spec not tested) Planned but not implemented: 4 requirement(s) in hw/ip/hmac/data/hmac.hjson > hmac > STATUS — The hmac scoreboard predicts status.fifo_empty, fifo_full and fifo_depth from a model of the FIFO write and read counts. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:383-387`, `hw/ip/hmac/dv/env/hmac_scoreboard.sv:510-514`)
- `f_0a9a9305dca6` (spec not tested) Planned but not implemented: 5 requirement(s) in hw/ip/hmac/data/hmac.hjson > hmac > alert_list … (3 sections) — All five items have implemented DV tests with checks. (`hw/ip/hmac/dv/hmac_sim_cfg.hjson:33`, `hw/ip/hmac/dv/hmac_sim_cfg.hjson:31`)
- `f_0ab899e6c3f2` (spec not tested) Planned but not implemented: 1 requirement(s) in hw/ip/hmac/doc/interfaces.md > Hardware Interfaces > Interrupts — The HMAC DV scoreboard models the fifo_empty interrupt, including the condition that the FIFO must have been full first, and compares it

against intr_state reads. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:693-697`, `hw/ip/hmac/dv/env/hmac_scoreboard.sv:389-392`)

- `f_17629afe1b7c` (spec not tested) Planned but not implemented: 5 requirement(s) in hw/ip/hmac/doc/interfaces.md > Hardware Interfaces … (3 sections) — All five items are covered by implemented checks. (`hw/ip/hmac/dv/sva/hmac_bind.sv:7-9`, `hw/ip/hmac/dv/hmac_sim_cfg.hjson:30`)
- `f_3a4bbd9db035` (spec not tested) Planned but not implemented: 1 requirement(s) in hw/ip/hmac/doc/programmers_guide.md > Programmer's Guide > FIFO Depth and Empty status — The HMAC DV environment checks this behaviour. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:693-697`, `hw/ip/hmac/dv/env/hmac_scoreboard.sv:391-392`)
- `f_872724e2676a` (spec not tested) Planned but not implemented: 5 requirement(s) in hw/ip/hmac/doc/programmers_guide.md > Programmer's Guide > Updating the configurations … (3 sections) — The finding treats these items as plan-only, but the HMAC DV environment implements them. (`hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:398-399`, `hw/ip/hmac/dv/env/hmac_scoreboard.sv:319-325`)
- `f_0b4237753b83` (spec not tested) Only weak evidence (non-failing checks, coverage, list entries): 1 requirement(s) in hw/ip/hmac/doc/theory_of_operation.md > Theory of Operation > Design Details > HMAC computation — The smoke vseq randomizes key_swap and writes it to CFG. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:862-866`)
- `f_2fc38343b1ac` (spec not tested) Register reset values: no suite covers every register — A shared generic CSR test library provides csr_hw_reset and csr_mem_rw_with_rand_reset, enabled per block via csr_tests_mode. (`hw/dv/tools/dvsim/tests/csr_tests.hjson:21-23`, `hw/dv/sv/csr_utils/csr_hw_reset_seq.sv:47-51`)
- `f_e684b2415bb9` (spec not tested) Register access: no suite covers every register — The finding does not name its block. (`hw/dv/tools/dvsim/tests/csr_tests.hjson:29-40`, `hw/ip/hmac/dv/hmac_sim_cfg.hjson:30`)

Withdrawn items from the groups in section 5 (96):

- Only the key registers relevant to the configured key length are consumed (e.g. KEY_0-KEY_7 for a 256-bit key). — Smoke randomizes all 32 key words and the scoreboard reference uses only the configured-length slice, so consuming extra words would fail the check. (`hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:26`)
- HMAC supports key lengths of 128, 256, 384, 512 and 1024 bits. — Smoke randomizes key_length over the one-hot values with scoreboard checking. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:878-880`)
- While the engine computes the hash, it discards attempts to update the secret key registers. — Random key writes mid-hash are dropped in the scoreboard model, and the final digest is compared with the original-key prediction. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:322-328`)
- SwInvalidConfig is reported for an invalid key length in HMAC mode. — The smoke test randomizes illegal key lengths (Key_None) in HMAC mode. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:173-174`)
- prim_packer compacts written data into word size and writes it to the MSG_FIFO when the write is not a full word write. — The sequence randomizes contiguous byte masks on TL writes, so sub-word writes are exercised and the digest is checked. (`hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:128-130`)
- HMAC supports key lengths of 128/256/384/512/1024 bits provided the key length does not exceed the digest block size (max 512-bit key for SHA-2 256). — The scoreboard predicts SwInvalidConfig for SHA-2 256 with a 1024-bit key, and the smoke sequence handles that case. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:173-177`)

- csr_rw — hmac_csr_rw runs via the imported csr_tests.hjson. (`hw/dv/tools/dvsim/tests/csr_tests.hjson:29`)
- alert_test — hmac_alert_test is run via the imported alert_test.hjson. (`hw/dv/tools/dvsim/tests/alert_test.hjson:13`)
- regwen_csr_and_corresponding_lockable_csr — The testplan maps it to csr_rw and csr_aliasing, which run for hmac. (`hw/dv/tools/dvsim/testplans/csr_testplan.hjson:100`)
- csr_aliasing — The generated hmac_csr_aliasing test exists through the csr_tests.hjson import. (`hw/dv/tools/dvsim/tests/csr_tests.hjson:45-47`)
- fifo_empty_status_interrupt — The plan item lists hmac_smoke and other tests, and the scoreboard checks the fifo_empty interrupt. (`hw/ip/hmac/data/hmac_testplan.hjson:113-120`)
- For SHA-2 384 the digest is read from DIGEST_0-DIGEST_12. — The scoreboard does check SHA-384 digest readout, comparing words 0-11 and skipping the rest. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:491`)
- The digest is cleared when the SHA-2 engine is disabled. — The scoreboard predicts a cleared digest and the smoke sequence reads it back and compares. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:306-309`)
- key_length 6'b00_1000 selects a 512-bit key. — Randomized smoke covers Key_512 and the CFG read-back is checked. (`hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:114-125`)
- Invalid or unsupported digest_size values are mapped to SHA2_None. — Smoke randomizes illegal digest_size values, writes them, and reads back against a scoreboard prediction of SHA2_None. (`hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:114-125`)
- key_length 6'b00_0001 selects a 128-bit key. — One-hot randomized key_length covers Key_128, the key_length read-back is checked, and the digest is checked. (`hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:114-125`)
- When sha_en is 0, the SHA engine does not initiate compression. — The scoreboard expects an error and no compression when sha_en=0 (the check_error_code calls in trigger_hash and wr_msg exercise it). (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:212-215`)
- With key_swap 0, KEY_* CSRs use big-endian representation. — The scoreboard model uses the key unswapped when key_swap=0 and compares the DUT digest against it. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:867-869`)
- A message written to MSG_FIFO one byte at a time is not affected by endian_swap. — Random write masks include single-byte writes, and the scoreboard compares the resulting digest. (`hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:358`)
- key_length is a 6-bit one-hot encoded field configuring the HMAC key length. — The key_length one-hot field is randomised, and the scoreboard computes the expected digest for each key length and compares it. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:876-889`)
- key_length 6'b00_0100 selects a 384-bit key. — The scoreboard model has an explicit Key_384 branch for each digest size, and the smoke test randomises over it. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:876-889`)
- digest_size is a 4-bit one-hot encoded field selecting the digest size for HMAC or SHA-2. — The scoreboard selects the SHA-256, SHA-384 or SHA-512 model by digest_size and compares the digest. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:876-889`)
- key_length 6'b01_0000 selects a 1024-bit key. — The scoreboard has Key_1024 branches for the SHA-384 and SHA-512 models, and the DV skips Key_1024 only for SHA-256 as an invalid configuration. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:876-889`)
- The key length determines how many KEY registers are relevant for the HMAC operation. — Scoreboard digest prediction depends on key_length and is checked on reads.

(`hw/ip/hmac/dv/env/hmac_scoreboard.sv:877-880`)

- DIGEST.digest: hardware receives a write-qualifier pulse on software writes (true). — Digest writes in the SAR flow are predicted by the scoreboard, and the restored digest is compared on later reads. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:336-340`)
- MSG_LENGTH_LOWER.v: the register is implemented outside the register block (true). — The length register is written and read back, and the DV_CHECK_EQ at line 726 fails if it is wrong. (`hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:726`)
- MSG_LENGTH_UPPER.v: hardware receives a write-qualifier pulse on software writes (true). — msg_length_upper is written with toggling 32-bit upper values and the result is checked at line 726. (`hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:726`)
- STATUS.hmac_idle: the register is implemented outside the register block (true). — check_idle compares the idle pin and predicts STATUS.hmac_idle, and the status read is checked. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:1060-1066`)
- KEY.key: hardware receives a write-qualifier pulse on software writes (true). — KEY writes drive the HMAC computation, and the digest comparison would fail if the write-qualifier pulse were wrong. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:496`)
- KEY.key: the register is implemented outside the register block (true). — KEY is written and its effect is checked through the digest comparison. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:496`)
- DIGEST.digest: the register is implemented outside the register block (true). — Digest registers are read and compared with the expected digest in the scoreboard. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:496`)
- SwPushMsgWhenDisallowed is reported when SW updates MSG_FIFO with more message words after CMD.process has been received (engine triggered to finalize). — The scoreboard predicts code 5 for message pushes when not allowed, and hmac_error_vseq provides the stimulus. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:147-148`)
- The digest is cleared when CFG.sha_en transitions from 1 to 0. — The scoreboard predicts a zeroed digest on sha_en 1->0 and checks later digest reads against it. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:306-308`)
- Once the FIFO is full, software cannot push more data until the current block is processed. — The scoreboard predicts FIFO fullness, with drain after block processing, and compares it with the DUT's status and the FIFO-empty interrupt. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:821-825`)
- MSG_FIFO occupies a wide address range rather than a single 4-byte address. — Random writes across the full window are scoreboarded and the resulting digest is checked. (`hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:104`)
- The byte order of the key registers is big-endian by default. — HMAC scoreboard computes the expected digest with the key kept big-endian when key_swap=0, and key_swap is randomized including 0. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:860-869`)
- With Key_None, an error is signalled to SW when start is attempted. — Random smoke hits Key_None with hmac_en, and the scoreboard checks the error and idle prediction. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:173-177`)
- Unsupported, invalid and all-zero key_length values are mapped to Key_None (6'b10_0000). — Illegal and zero key_length are randomized and the Key_None mapping is predicted and checked on read. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:359-373`)
- key_length 6'b00_0010 selects a 256-bit key. — Random key_length includes Key_256 (the default is 6'b000010). (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:359-373`)
- Hardware byte swaps for endian_swap are performed at TL-UL word granularity. — Random sub-word writes with endian_swap are checked against a per-word swap model.

(`hw/ip/hmac/dv/env/hmac_scoreboard.sv:157-163`)

- Unsupported, invalid and all-zero digest_size values are mapped to SHA2_None (4'b1000). — Illegal digest_size is randomized and the SHA2_None mapping is predicted. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:359-373`)
- When endian_swap is 1, each multi-byte value written to MSG_FIFO, regardless of alignment, is added to the message in big-endian byte order. — Unaligned and sub-word writes are randomized and the scoreboard models big-endian order for them. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:157-163`)
- The fatal_fault alert is triggered when a fatal TL-UL bus integrity fault is detected. — The common tl_intg_err and alert_test tests run for HMAC via hmac_sim_cfg. (`hw/ip/hmac/dv/hmac_sim_cfg.hjson:33`)
- The HMAC implements an end-to-end bus integrity scheme as the BUS.INTEGRITY countermeasure (HMAC.BUS.INTEGRITY). — The BUS.INTEGRITY sec_cm plan item maps to hmac_tl_intg_err, which is run through the imported tl_access_tests. (`hw/ip/hmac/dv/hmac_sim_cfg.hjson:33`)
- CFG is updated only when the engine is idle. — The scoreboard ignores the CFG write during hashing and the vseq issues such writes. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:296-300`)
- In HMAC mode, start is blocked when key_length is Key_1024 and digest_size is SHA2_256. — The vseq flags Key_1024 with SHA2_256 as invalid and the scoreboard checks for SwInvalidConfig. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:198-199`)
- A software write to CFG while the engine is computing a hash is discarded. — The write_discard_config stimulus and the scoreboard no-update handling cover the discard. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:296-300`)
- The fifo_empty interrupt indicates that the message FIFO is empty. — The scoreboard predicts the fifo_empty interrupt and checks it through intr_state reads in the smoke test. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:693-697`)
- When CFG.key_swap is 1, the endianness of each KEY_* register is swapped (to little endian). — The scoreboard models the key swap and compares the predicted digest, with key_swap randomized in the smoke stimulus. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:862-866`)
- The hmac module has a TL-UL bus device interface named tl. — The bound tlul_assert and the csr tests run on the tl port. (`hw/ip/hmac/dv/sva/hmac_bind.sv:7-9`)
- The tl port is a req_rsp interface of type tlul_pkg::tl acting as responder with width 1. — The bound tlul_assert checks the device-side tl_i/tl_o req_rsp port. (`hw/ip/hmac/dv/sva/hmac_bind.sv:7-9`)
- The fifo_empty interrupt is raised only if the message FIFO has previously been full. — The scoreboard requires fifo_full_detected and an empty FIFO before predicting the interrupt, and compares it with intr_state. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:693`)
- The fifo_empty interrupt is not raised once software has written the Process or Stop command to finish the hashing operation. — The scoreboard clears fifo_full_detected on a Process or Stop command, so no interrupt is predicted afterwards. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:678-680`)
- The fifo_empty interrupt is raised only if the message FIFO is actually writable by software (all listed conditions are met). — The scoreboard's prediction logic implements the writability conditions and its predicted intr_state is checked on reads. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:693`)
- CMD.hash_process: hardware receives a write-qualifier pulse on software writes (true). — hwqe is an implementation attribute. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:171-193`)
- CMD.hash_start: hardware receives a write-qualifier pulse on software writes (true). — hwqe is an implementation attribute. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:171-193`)
- CMD.hash_stop: the register is implemented outside the register block (true). — hwext is an implementation attribute. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:411-419`)

- STATUS.fifo_empty: the register is implemented outside the register block (true). — The scoreboard models fifo_empty and the generic CSR read check compares the modelled value to the value read from the status register. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:383-387`)
- STATUS.fifo_full: the register is implemented outside the register block (true). — The scoreboard models fifo_full and the generic CSR read check compares the modelled value to the value read from the status register. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:383-387`)
- The fifo_empty status interrupt fires when the Message FIFO runs empty after the SHA-2 engine resumes popping data. — The DV scoreboard models and checks the fifo_empty interrupt in intr_state. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:693-697`)
- The message FIFO buffer is 32 entries deep and 32 bits wide. — The write sequence waits on status.fifo_depth against a depth of 32, and the scoreboard checks fifo_depth and fifo_full reads. (`hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:449-451`)
- In HMAC mode the secret key length is configurable up to 1024 bits. — Random configs and test vectors cover key lengths up to 1024 in HMAC mode, and the scoreboard compares the digest. (`hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:804-807`)
- 1024-bit keys are not supported for SHA-2 256 (block size 512-bit). — The scoreboard predicts the invalid config for SHA-2 256 with a 1024-bit key, and the vseqs drive that combination. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:173-177`)
- csr_hw_reset — hmac_csr_hw_reset exists via csr_tests.hjson (`hw/dv/tools/dvsim/tests/csr_tests.hjson:20-24`)
- back_pressure — hmac_back_pressure test with zero_delays exists (`hw/ip/hmac/dv/hmac_sim_cfg.hjson:73-75`)
- csr_mem_rw_with_rand_reset — hmac_csr_mem_rw_with_rand_reset exists via csr_tests.hjson (`hw/dv/tools/dvsim/tests/csr_tests.hjson:61-63`)
- MSG_LENGTH_UPPER.v: the register is implemented outside the register block (true). — The register is read and written in tests, and the scoreboard predicts and checks its value. (`hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:700-707`)
- For SHA-2 384/512 the message length is 128 bits, with the upper 64 bits zero-padded: {32'b0, 32'b0, MSG_LENGTH_UPPER, MSG_LENGTH_LOWER}. — Every SHA-384/512 digest is compared against the DPI model and the NIST vectors, which only match if the upper 64 length bits are zero. (`hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:700-707`)
- Attempts to update CFG or KEY while processing are discarded. — Random CFG/KEY writes during hash, discarded in scoreboard. (`hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:398-399`)
- CFG and KEY registers cannot be updated while the engine is processing messages. — Same stimulus and model. (`hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:398-399`)
- KEY register ignores any attempt to access the secret key during processing. — KEY write during hash discarded and flagged. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:319-325`)
- For keyed HMAC, the values of KEY_0..X must also be maintained and restored as part of the saved context, where X is the last register used for the key length (e.g. X=7 for HMAC-256). — S&R in the base vseq keeps key_bak and probes secret_key; digest checked by the scoreboard. (`hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:398-399`)
- SwPushMsgWhenDisallowed is reported when SW pushes message words before the engine has been triggered to start. — Scoreboard expects the error for pushes before start. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:147-148`)
- CFG.hmac_en: the register is implemented outside the register block (true). — The scoreboard predicts behaviour from ral.cfg.hmac_en, so a wrong implementation would fail the digest check. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:173`)

- CFG.digest_size: hardware receives a write-qualifier pulse on software writes (true). — The scoreboard predicts behaviour from ral.cfg.digest_size, which is written through the base vseq's hmac_init, so a wrong write path would fail the digest check. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:173`)
- CFG.key_length: the register is implemented outside the register block (true). — The scoreboard uses ral.cfg.key_length in its invalid-configuration check and digest prediction. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:173`)
- CFG.endian_swap: the register is implemented outside the register block (true). — The scoreboard applies endian_swap when it builds the expected message, so the digest comparison fails if the swap is wrong. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:160`)
- CFG.key_swap: the register is implemented outside the register block (true). — The base vseq randomizes key_swap and the coverage crosses sample it. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:173`)
- Unknown checks on DUT outputs — ASSERT_KNOWN assertions in hmac.sv check this in every simulation. (`hw/ip/hmac/rtl/hmac.sv:1087-1088`)
- tl_intg_err — hmac_tl_intg_err comes from the imported common tl_access_tests list. (`hw/dv/tools/dvsim/tests/tl_access_tests.hjson:20-23`)
- csr_bit_bash — hmac_csr_bit_bash comes from the imported common csr_tests list. (`hw/dv/tools/dvsim/tests/csr_tests.hjson:37-39`)
- CMD.hash_stop: hardware receives a write-qualifier pulse on software writes (true). — hash_stop is written by trigger_hash_stop, and the scoreboard checks the resulting stop and digest behaviour. (`hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:248-250`)
- CMD.hash_process: the register is implemented outside the register block (true). — hash_process is written and its effect is checked through the digest and the done interrupt. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:182-188`)
- CMD.hash_continue: the register is implemented outside the register block (true). — hash_continue is written by trigger_hash_continue and its effect is checked in the scoreboard. (`hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:233-235`)
- CMD.hash_start: the register is implemented outside the register block (true). — hash_start is written in every hash test, and the digest is checked for each. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:182-188`)
- CMD.hash_continue: hardware receives a write-qualifier pulse on software writes (true). — hash_continue only takes effect in hardware through the write-qualifier pulse. (`hw/ip/hmac/dv/env/seq_lib/hmac_base_vseq.sv:233-235`)
- The HMAC key length must not exceed the block size of the configured SHA-2 digest: at most 1024 bits for SHA-2 384/512 and at most 512 bits for SHA-2 256. — The scoreboard checks the invalid-config error, and random smoke stimulus generates SHA-256 with a 1024-bit key. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:173-177`)
- STATUS.fifo_depth reports the number of entry slots currently occupied in the FIFO (the FIFO entry count). — The scoreboard compares the read fifo_depth against the model and errors on a mismatch. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:510-514`)
- STATUS.fifo_full indicates that the Message FIFO is full. — fifo_full is predicted in the status data and compared on read. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:383-387`)
- fifo_empty indicates the FIFO is empty. — fifo_empty is predicted in the status data and compared on read. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:383-387`)
- STATUS.fifo_depth: the register is implemented outside the register block (true). — The externally implemented fifo_depth field is read back and checked by the scoreboard. (`hw/ip/hmac/dv/env/hmac_scoreboard.sv:510-514`)

- FPV_ASSERTIONS_PROVEN_AT_V3 — Checklist item marked N/A, not something a test implements. (`hw/ip/hmac/doc/checklist.md:245`)
- DESIGN_DELTAS_CAPTURED_V3 — Documentation checklist item marked Done, not a test item. (`hw/ip/hmac/doc/checklist.md:243`)
- TLUL assertions — tlul_assert is bound in hmac_bind.sv, so protocol checks run in every sim. (`hw/ip/hmac/dv/sva/hmac_bind.sv:7-14`)
- SIM_NIGHTLY_REGRESSION_AT_V3 — Regression-process checklist item, documented in an issue, not a test item. (`hw/ip/hmac/doc/checklist.md:246`)
- sec_cm_bus_integrity — hmac_tl_intg_err comes from the shared tl_access_tests.hjson that HMAC imports. (`hw/dv/tools/dvsim/tests/tl_access_tests.hjson:20-23`)

Run record

Stage	Model
Spec extraction	claude-sonnet-5-5
Test extraction	claude-sonnet-5-5
Merge and link	claude-sonnet-5-5
Correlate	claude-sonnet-5-5
Analyze	claude-opus-5-5, claude-sonnet-5-5
Verify	claude-sonnet-5-5

This run: $1.94 in 210 s (cached results reused where the inputs were unchanged). All LLM spend recorded for this workspace: $32.25.

Manifest `work/opentitan-hmac/manifest.yaml` (sha256 `79bf9387ac9e`); findings `work/opentitan-hmac/findings.yaml`; verifier transcripts in `work/opentitan-hmac/verify_runs/<finding id>/`.